

Please find your DSA Sheet Below All the best for your Preparation

<https://livecohortbypradeep.com/>

LLD WEEKEND BATCH LAUNCH

BUILD STRONG DESIGNS. CRACK BETTER INTERVIEWS.

- 15+ LLD PROBLEMS SOLVED LIVE
- DESIGN PATTERNS DEEP DIVE
- 2-3 MOCK INTERVIEW SESSIONS

STARTING **29TH AUGUST**

SAT & SUN • 10:30 AM - 12 PM IST

DESIGN SOLVE SUCCEED

GOOD CODE WORKS. GREAT DESIGN LASTS.

JOIN THE BATCH. LEVEL UP YOUR DESIGN GAME!

```
class Design {
  public:
    void solve();
};
```

LLD

DSA Patterns Mastery

75+ Coding Interview Problems, Solved Pattern-by-Pattern

Every problem includes: how to recognize the pattern, brute-force approach, optimized approach, pseudocode, and time/space complexity for both.

Compiled by Pradeep Kumar

Work in progress — built pattern by pattern, batch by batch

Recommended Resources — Watch Before You Read

For every pattern in this document, here are a few good YouTube videos to watch first — they'll help the written explanations click faster.

Two Pointers

- [Two Pointers in 7 minutes | LeetCode Pattern](#)
- [L1. Two Pointers || 25 DSA Patterns || New & Easy Way to Master DSA](#)
- [Episode 02: Master DSA Patterns with the Two Pointer Technique](#)
- [DSA Interview Question: 2 Pointer Technique | Two Sum LeetCode Problem](#)

Sliding Window

- [Sliding Window in 7 minutes | LeetCode Pattern](#)
- [L1. Introduction to Sliding Window and 2 Pointers | Templates | Patterns](#)
- [Sliding Window Pattern | Fixed & Variable Size | DSA Tutorial](#)
- [Sliding Window Maximum | Deque Data Structure | DSA-One Course #54](#)

Fast & Slow Pointers

- [Fast and Slow Pointers in 6 Minutes | LeetCode Pattern](#)
- [Linked List Cycle — Slow & Fast Pointers | LeetCode 141 \(Python\)](#)
- [Find Middle of Linked List — Fast & Slow Pointer \(Tortoise and Hare\)](#)
- [Linked List Cycle II \(Fast and Slow Pointers Coding Pattern\)](#)

Cyclic Sort

- [Array - 43: Cyclic Sort | Sort the Elements from 1 to n](#)
- [Missing Number using Cyclic Sort \(in-depth explanation\)](#)
- [Coding Interview Pattern: Cyclic Sort](#)
- [Cracking the Cyclic Sort Interview Pattern \(Part 1\)](#)

Merge Intervals

- [Merge Interval Algorithm in 4 Minutes](#)
- [Coding Pattern - Merge Interval](#)
- [Coding Interview Pattern | Merge Intervals](#)
- [DSA Patterns L31 | Master Merge Interval DSA Patterns](#)

Modified Binary Search

- [Search in Rotated Sorted Array | Binary Search | LeetCode 33](#)
- [BS-4. Search Element in Rotated Sorted Array - I](#)
- [Modified Binary Search — Find Minimum in Rotated Sorted Array](#)
- [Search an Element in a Sorted & Rotated Array | Binary Search Part 3](#)

Tree BFS

- [Breadth First Search \(BFS\) Pattern: 1/3 \(Binary Tree Level Order Traversal\)](#)
- [Binary Tree Level Order Traversal \(BFS\) — LeetCode 102 — Trees \(Python\)](#)
- [L8. Level Order Traversal of Binary Tree | BFS | C++ | Java](#)
- [Level Order Traversal Explained: BFS Tree Traversal for Beginners](#)

Backtracking (Subsets & Permutations)

- [Backtracking Episode 1: LeetCode Permutation and Subsets](#)

- [BACKTRACKING TEMPLATE | How to Solve LeetCode Backtracking Questions](#)
- [Backtracking: Permutations & Combinations Explained | Geekific](#)
- [Backtracking: Permutation, Combination, Subset Problems Series](#)

Top K Elements (Heap)

- [Top K Elements in 6 minutes | LeetCode Pattern](#)
- [Solve Any HEAP Question By This Template | Top K Frequent Elements](#)
- [L11 & L13. Heap — Min & Max Heaps \(Priority Queue\) — Where Heap is Applied: Top K Elements](#)
- [Top K Frequent Elements — Heap vs Bucket Sort Explained](#)

Monotonic Stack

- [Monotonic Stack in 6 minutes | LeetCode Pattern](#)
- [Monotonic Stack Data Structure Explained](#)
- [Next Greater Element — Monotonic Stack Algorithm Tutorial](#)
- [Monotonic Stack Explained: Solve Tricky Coding Problems Easily!](#)

Contents

Two Pointers

- Q1. Two Sum II — Input Array Is Sorted
- Q2. Valid Palindrome
- Q3. Container With Most Water
- Q4. 3Sum
- Q5. Remove Duplicates from Sorted Array

Sliding Window

- Q6. Maximum Sum Subarray of Size K (Fixed Window)
- Q7. Longest Substring Without Repeating Characters
- Q8. Minimum Size Subarray Sum (Variable Window)
- Q9. Longest Repeating Character Replacement
- Q10. Find All Anagrams in a String

Fast & Slow Pointers

- Q11. Middle of the Linked List
- Q12. Linked List Cycle
- Q13. Linked List Cycle II (Find the Start of the Cycle)
- Q14. Happy Number
- Q15. Palindrome Linked List
- Q16. Reorder List
- Q17. Circular Array Loop

Cyclic Sort

- Q18. Cyclic Sort (the base pattern)
- Q19. Find the Missing Number
- Q20. Find All Missing Numbers
- Q21. Find the Duplicate Number
- Q22. Find All Duplicates in an Array
- Q23. Set Mismatch
- Q24. First Missing Positive

Merge Intervals

- Q25. Merge Intervals
- Q26. Insert Interval
- Q27. Non-overlapping Intervals (Erase Overlap Intervals)
- Q28. Meeting Rooms II
- Q29. Interval List Intersections
- Q30. Minimum Number of Arrows to Burst Balloons
- Q31. Employee Free Time
- Q32. My Calendar I

Modified Binary Search

- Q33. Binary Search (the base pattern)
- Q34. Find First and Last Position of Element in Sorted Array
- Q35. Search in Rotated Sorted Array
- Q36. Find Minimum in Rotated Sorted Array
- Q37. Search a 2D Matrix
- Q38. Find Peak Element

[Q39. Koko Eating Bananas \(Binary Search on the Answer\)](#)

[Q40. Capacity To Ship Packages Within D Days \(Binary Search on the Answer\)](#)

Tree BFS

[Q41. Binary Tree Level Order Traversal](#)

[Q42. Binary Tree Zigzag Level Order Traversal](#)

[Q43. Average of Levels in Binary Tree](#)

[Q44. Binary Tree Right Side View](#)

[Q45. Minimum Depth of Binary Tree](#)

[Q46. Populating Next Right Pointers in Each Node](#)

[Q47. Cousins in Binary Tree](#)

Backtracking (Subsets & Permutations)

[Q48. Subsets](#)

[Q49. Subsets II \(With Duplicates\)](#)

[Q50. Permutations](#)

[Q51. Combinations](#)

[Q52. Combination Sum](#)

[Q53. Word Search](#)

[Q54. Generate Parentheses](#)

Top K Elements (Heap)

[Q55. Kth Largest Element in an Array](#)

[Q56. Top K Frequent Elements](#)

[Q57. K Closest Points to Origin](#)

[Q58. Kth Largest Element in a Stream](#)

[Q59. Find K Pairs with Smallest Sums](#)

[Q60. Top K Frequent Words](#)

[Q61. Kth Smallest Element in a Sorted Matrix](#)

[Q62. Task Scheduler](#)

Monotonic Stack

[Q63. Next Greater Element I](#)

[Q64. Next Greater Element II \(Circular Array\)](#)

[Q65. Daily Temperatures](#)

[Q66. Largest Rectangle in Histogram](#)

[Q67. Trapping Rain Water](#)

[Q68. Remove K Digits](#)

[Q69. Online Stock Span](#)

[Q70. Asteroid Collision](#)

Total problems in this document so far: 70

Pattern: Two Pointers

How to identify this pattern

- The input is a sorted array/list, or can be sorted without losing what you need.
- You are looking for a pair, triplet, or subrange that satisfies a sum/difference/product condition.
- You need to compare elements from both ends of a structure moving inward (or two indices moving in the same direction at different speeds).
- A brute-force solution uses two nested loops ($O(n^2)$) to check every pair — that is almost always your signal that two pointers can cut it to $O(n)$.
- Keywords to watch for: "sorted array", "pair with sum", "palindrome", "remove duplicates in place", "container/area between two lines".

Q1. Two Sum II — Input Array Is Sorted

Problem Statement

Given a 1-indexed array of integers 'numbers' that is already sorted in non-decreasing order, find two numbers such that they add up to a specific 'target' number. Return the indices of the two numbers (1-indexed) as an array [index1, index2], where index1 < index2. You may assume exactly one solution exists, and you may not use the same element twice.

Example numbers = [2, 7, 11, 15], target = 9 → Output: [1, 2] (numbers[1] + numbers[2] = 2 + 7 = 9)

 **Pattern Insight — how to spot it:** The array is already sorted — that's the big giveaway. Because it's sorted, if the sum of the two boundary elements is too small, the only way to increase it is to move the left pointer right (to a bigger number). If it's too big, move the right pointer left (to a smaller number). This monotonic behavior is exactly what lets two pointers replace nested loops.

Brute Force Approach

Check every pair of indices (i, j) with $i < j$ and test if $\text{numbers}[i] + \text{numbers}[j] == \text{target}$. Sorting isn't even used here.

```
for i from 0 to n-1:
    for j from i+1 to n-1:
        if numbers[i] + numbers[j] == target:
            return [i+1, j+1]
```

 **Time:** $O(n^2)$  **Space:** $O(1)$

Optimized Approach

Place a 'left' pointer at index 0 and a 'right' pointer at index n-1. Compute the sum. If it equals target, return. If it's less than target, move left forward (increase sum). If it's more than target, move right backward (decrease sum). Repeat until left meets right.

```
left = 0, right = n - 1
while left < right:
    sum = numbers[left] + numbers[right]
    if sum == target:
        return [left+1, right+1]
    else if sum < target:
        left = left + 1
    else:
        right = right - 1
```

 **Time:** $O(n)$  **Space:** $O(1)$

Q2. Valid Palindrome

Problem Statement

Given a string 's', determine if it is a palindrome, considering only alphanumeric characters and ignoring case.

Example s = "A man, a plan, a canal: Panama" → true s = "race a car" → false

 **Pattern Insight — how to spot it:** Comparing a string to its reverse from both ends is the textbook two-pointer shape: one pointer starts at the beginning, one at the end, and they walk toward the middle checking a condition at each step. Any time you need to check symmetry around a center, think two pointers first.

Brute Force Approach

Build a cleaned string containing only lowercase alphanumeric characters, then compare it to its own reverse.

```
cleaned = ""
for ch in s:
    if ch.isalnum():
        cleaned += ch.lower()
reversed = cleaned[::-1]
return cleaned == reversed
```

 **Time:** O(n)  **Space:** O(n) // extra string(s) built

Optimized Approach

Use left and right pointers on the original string. Skip non-alphanumeric characters on either side. Compare characters (case-insensitively); mismatch means not a palindrome. Move both pointers inward until they cross.

```
left = 0, right = len(s) - 1
while left < right:
    while left < right and not s[left].isalnum():
        left += 1
    while left < right and not s[right].isalnum():
        right -= 1
    if s[left].lower() != s[right].lower():
        return False
    left += 1
    right -= 1
return True
```

 **Time:** O(n)  **Space:** O(1) // no extra string built

Q3. Container With Most Water

Problem Statement

Given an array 'height' of n non-negative integers where each represents a vertical line at position i, find two lines that together with the x-axis form a container that holds the most water. Return the maximum area.

Example height = [1,8,6,2,5,4,8,3,7] → Output: 49



Pattern Insight — how to spot it: Area = width x min(height[left], height[right]). Start with the widest possible container (both pointers at the ends). The width can only shrink as pointers move inward, so the only way to potentially get a bigger area is to increase the limiting (shorter) height. That's why you always move the pointer at the SHORTER line — moving the taller one can never help, since width shrinks and the shorter side still caps the area.

Brute Force Approach

Try every pair of lines (i, j), compute the area for each, and keep the maximum.

```
maxArea = 0
for i from 0 to n-1:
    for j from i+1 to n-1:
        area = (j - i) * min(height[i], height[j])
        maxArea = max(maxArea, area)
return maxArea
```

⌚ **Time:** $O(n^2)$ 📦 **Space:** $O(1)$

Optimized Approach

left = 0, right = n-1. Compute area, update max. Move the pointer pointing to the shorter line inward (since it's the bottleneck). Repeat until pointers meet.

```
left = 0, right = n - 1
maxArea = 0
while left < right:
    area = (right - left) * min(height[left], height[right])
    maxArea = max(maxArea, area)
    if height[left] < height[right]:
        left = left + 1
    else:
        right = right - 1
return maxArea
```

⌚ **Time:** $O(n)$ 📦 **Space:** $O(1)$

Q4. 3Sum

Problem Statement

Given an integer array 'nums', return all unique triplets [nums[i], nums[j], nums[k]] such that $i \neq j \neq k$ and $nums[i] + nums[j] + nums[k] == 0$. The solution set must not contain duplicate triplets.

Example nums = [-1,0,1,2,-1,-4] → Output: [[-1,-1,2], [-1,0,1]]

 **Pattern Insight — how to spot it:** This is 'Two Sum' one level up: fix one number, then use the two-pointer technique on the REMAINING sorted array to find pairs that sum to the negative of the fixed number. Sorting first is what unlocks two pointers here, and it also makes duplicate-skipping trivial (equal values sit next to each other).

Brute Force Approach

Three nested loops checking every triplet, using a set to dedupe the sorted triplets found.

```
result = empty set
for i from 0 to n-1:
    for j from i+1 to n-1:
        for k from j+1 to n-1:
            if nums[i]+nums[j]+nums[k] == 0:
                triplet = sort([nums[i], nums[j], nums[k]])
                result.add(triplet)
return list(result)
```

 **Time:** $O(n^3)$  **Space:** $O(n)$ // for the result set

Optimized Approach

Sort nums. For each index i (skipping duplicates), run the two-pointer search on the subarray (i+1 .. n-1) for pairs summing to $-nums[i]$. Skip duplicate values for left/right too, to avoid duplicate triplets.

```
sort(nums)
result = []
for i from 0 to n-1:
    if i > 0 and nums[i] == nums[i-1]:
        continue // skip duplicate anchor
    left = i + 1, right = n - 1
    while left < right:
        sum = nums[i] + nums[left] + nums[right]
        if sum == 0:
            result.append([nums[i], nums[left], nums[right]])
            left = left + 1
            right = right - 1
            while left < right and nums[left] == nums[left-1]:
                left = left + 1
            while left < right and nums[right] == nums[right+1]:
                right = right - 1
        else if sum < 0:
            left = left + 1
        else:
            right = right - 1
    return result
```

 **Time:** $O(n^2)$ // $O(n \log n)$ sort + $O(n) \times O(n)$ two-pointer scan  **Space:** $O(1)$ to $O(n)$ depending on sort implementation (excluding output)

Q5. Remove Duplicates from Sorted Array

Problem Statement

Given an integer array 'nums' sorted in non-decreasing order, remove the duplicates in place such that each unique element appears only once. Return the number of unique elements 'k'; the first k elements of nums should hold the final result.

Example nums = [0,0,1,1,1,2,2,3,3,4] → k = 5, nums becomes [0,1,2,3,4,...]



Pattern Insight — how to spot it: This is the 'slow/fast pointer' variant of two pointers, used for in-place array compaction. The slow pointer marks the boundary of the 'clean' (deduplicated) region built so far; the fast pointer scans ahead looking for the next new value to bring into that region. Any 'remove/move elements in place while preserving order' problem is a strong signal for this pattern.

Brute Force Approach

Copy unique elements into a new array/list, then copy them back into nums.

```
temp = []
for i from 0 to n-1:
    if i == 0 or nums[i] != nums[i-1]:
        temp.append(nums[i])
for i from 0 to len(temp)-1:
    nums[i] = temp[i]
return len(temp)
```

Time: O(n) **Space:** O(n) // temp array

Optimized Approach

slow starts at 0 (the last confirmed-unique index). fast scans from 1 to n-1. Whenever nums[fast] != nums[slow], it's a new unique value: increment slow and copy it there. slow+1 at the end is the count of unique elements.

```
if n == 0: return 0
slow = 0
for fast from 1 to n-1:
    if nums[fast] != nums[slow]:
        slow = slow + 1
        nums[slow] = nums[fast]
return slow + 1
```

Time: O(n) **Space:** O(1) // in place, no extra array

Pattern: Sliding Window

How to identify this pattern

- You're working with a contiguous subarray or substring (never a subsequence with gaps).
- You need to find the max/min/count of something over 'all windows of size k' — that's a FIXED window.
- You need the longest/shortest contiguous range satisfying a condition (sum $\geq$ target, no repeating chars, at most k distinct, etc.) — that's a VARIABLE (dynamic) window.
- The brute-force is recomputing a sum/count from scratch for every start index ($O(n^2)$ or worse) — sliding window reuses work from the previous window instead of recomputing it, dropping this to $O(n)$.
- Keywords to watch for: "contiguous subarray/substring", "of size k", "longest/shortest substring with...", "at most/exactly k distinct".

Q6. Maximum Sum Subarray of Size K (Fixed Window)

Problem Statement

Given an integer array 'nums' and an integer 'k', find the maximum sum of any contiguous subarray of size exactly k.

Example nums = [2,1,5,1,3,2], k = 3 → Output: 9 (subarray [5,1,3])

 **Pattern Insight — how to spot it:** Fixed-size window: once a window of size k is built, sliding it forward by one only changes two elements — the one that leaves on the left and the one that enters on the right. There's no need to re-sum all k elements every time; just subtract the outgoing element and add the incoming one.

Brute Force Approach

For every starting index i, sum the k elements from i to i+k-1 from scratch, and track the maximum.

```
maxSum = -infinity
for i from 0 to n-k:
    windowSum = 0
    for j from i to i+k-1:
        windowSum = windowSum + nums[j]
    maxSum = max(maxSum, windowSum)
return maxSum
```

 **Time:** $O(n * k)$  **Space:** $O(1)$

Optimized Approach

Compute the sum of the first window (indices 0..k-1). Then slide: at each step, subtract the element leaving the window and add the element entering it. Track the maximum sum seen.

```
windowSum = sum(nums[0..k-1])
maxSum = windowSum
for i from k to n-1:
    windowSum = windowSum + nums[i] - nums[i-k]
    maxSum = max(maxSum, windowSum)
return maxSum
```

 **Time:** $O(n)$  **Space:** $O(1)$

Q7. Longest Substring Without Repeating Characters

Problem Statement

Given a string 's', find the length of the longest substring without repeating characters.

Example `s = "abcabcbb" → Output: 3 ("abc")` `s = "pwwkew" → Output: 3 ("wke")`

 **Pattern Insight — how to spot it:** Variable window: grow the right edge to include new characters; when a repeat is found inside the current window, shrink from the left until the repeat is gone. A hash set/map tracking what's currently 'in the window' is what makes checking for repeats $O(1)$ instead of re-scanning the whole window.

Brute Force Approach

Check every substring (i, j) and verify all its characters are unique, tracking the max length found.

```
maxLen = 0
for i from 0 to n-1:
    seen = empty set
    for j from i to n-1:
        if s[j] in seen:
            break
        seen.add(s[j])
        maxLen = max(maxLen, j - i + 1)
return maxLen
```

 **Time:** $O(n^2)$ // $O(n^3)$ if uniqueness is re-checked from scratch each time  **Space:** $O(\min(n, \text{charset}))$

Optimized Approach

left = 0, use a set 'window' for characters currently in range [left, right]. For each right, while `s[right]` is already in window, remove `s[left]` from window and move left forward. Add `s[right]` to window. Update maxLen with (right - left + 1) at each step.

```
left = 0
window = empty set
maxLen = 0
for right from 0 to n-1:
    while s[right] in window:
        window.remove(s[left])
        left = left + 1
    window.add(s[right])
    maxLen = max(maxLen, right - left + 1)
return maxLen
```

 **Time:** $O(n)$ // each character added and removed from window at most once  **Space:** $O(\min(n, \text{charset}))$

Q8. Minimum Size Subarray Sum (Variable Window)

Problem Statement

Given an array of positive integers 'nums' and a positive integer 'target', find the minimal length of a contiguous subarray whose sum is $\geq$ target. If no such subarray exists, return 0.

Example target = 7, nums = [2,3,1,2,4,3] → Output: 2 (subarray [4,3])

 **Pattern Insight — how to spot it:** All numbers are positive, so the window sum only ever increases when you expand right and only ever decreases when you shrink left — the sum behaves monotonically. That monotonic behavior is exactly what lets you safely shrink from the left as soon as the condition (sum $\geq$ target) is met, instead of exploring every possible window.

Brute Force Approach

For every start index i , extend j forward accumulating the sum until it's $\geq$ target, and record the window length. Repeat for every i .

```
minLen = infinity
for i from 0 to n-1:
    sum = 0
    for j from i to n-1:
        sum = sum + nums[j]
        if sum >= target:
            minLen = min(minLen, j - i + 1)
            break
return minLen if minLen != infinity else 0
```

 **Time:** $O(n^2)$  **Space:** $O(1)$

Optimized Approach

left = 0, windowSum = 0. Expand right, adding nums[right] to windowSum. Whenever windowSum $\geq$ target, record the window length and shrink from the left (subtracting nums[left], moving left forward) as long as the sum still stays $\geq$ target — this finds the smallest valid window ending near 'right'.

```
left = 0, windowSum = 0, minLen = infinity
for right from 0 to n-1:
    windowSum = windowSum + nums[right]
    while windowSum >= target:
        minLen = min(minLen, right - left + 1)
        windowSum = windowSum - nums[left]
        left = left + 1
return minLen if minLen != infinity else 0
```

 **Time:** $O(n)$ // left pointer moves forward at most n times total  **Space:** $O(1)$

Q9. Longest Repeating Character Replacement

Problem Statement

Given a string 's' and an integer 'k', you can replace at most k characters in the string with any other character. Return the length of the longest substring containing the same letter after performing at most k replacements.

Example s = "AABABBA", k = 1 → Output: 4 (replace one 'B' in "ABBA" to get "AAAA")

 **Pattern Insight — how to spot it:** A window is 'valid' when (window length - count of its most frequent character) ≤ k — that's the number of characters you'd need to replace. Track character frequencies inside the window with a hash map/array and keep a running maxFreq. Unlike problem 8, here the window never needs to shrink below its best-known length — you just slide it forward, which is a common variant of the variable-window pattern.

Brute Force Approach

For every substring, count the frequency of each character, find the max frequency, and check if (length - maxFreq) ≤ k.

```
maxLen = 0
for i from 0 to n-1:
    freq = empty map
    for j from i to n-1:
        freq[s[j]] += 1
        maxFreq = max value in freq
        if (j - i + 1) - maxFreq <= k:
            maxLen = max(maxLen, j - i + 1)
return maxLen
```

 **Time:** $O(n^2)$ // or $O(26 * n^2)$ counting max frequency each time  **Space:** $O(26) = O(1)$

Optimized Approach

left = 0, freq map for the window, maxFreq = highest count of any single character seen so far in any window. Expand right, update freq and maxFreq. If (window length - maxFreq) > k, shrink from the left by one. Track the max window length throughout (note: maxFreq is allowed to be 'stale' — it never causes an incorrect answer, only avoids unnecessary shrinking).

```
left = 0, maxFreq = 0
freq = map of char -> count, all 0
maxLen = 0
for right from 0 to n-1:
    freq[s[right]] += 1
    maxFreq = max(maxFreq, freq[s[right]])
    if (right - left + 1) - maxFreq > k:
        freq[s[left]] -= 1
        left = left + 1
    maxLen = max(maxLen, right - left + 1)
return maxLen
```

 **Time:** $O(n)$  **Space:** $O(26) = O(1)$ // fixed alphabet size

Q10. Find All Anagrams in a String

Problem Statement

Given two strings 's' and 'p', return the start indices of all anagrams of 'p' in 's'. An anagram is a rearrangement of all the letters of 'p' using each exactly once.

Example s = "cbaebabacd", p = "abc" → Output: [0, 6]

 **Pattern Insight — how to spot it:** This is a FIXED window of size len(p) sliding across s, where 'valid' means the character-frequency map of the current window exactly matches the character-frequency map of p. Because the window size never changes, you can maintain the match incrementally: when the window slides, one character leaves and one enters, and you only need to update the counts for those two characters, not recompute the whole comparison.

Brute Force Approach

For every window of length len(p) in s, build its frequency map from scratch and compare it to p's frequency map.

```
pFreq = frequency map of p
result = []
for i from 0 to len(s) - len(p):
    windowFreq = frequency map of s[i .. i+len(p)-1] // built from scratch
    if windowFreq == pFreq:
        result.append(i)
return result
```

 **Time:** $O(n * m)$ // $n = \text{len}(s)$, $m = \text{len}(p)$; rebuilding + comparing each window  **Space:** $O(m)$ // for frequency maps

Optimized Approach

Build pFreq (frequency map of p) once. Slide a window of size len(p) across s using a running sFreq map: add the entering character's count, remove the leaving character's count once the window exceeds size len(p). After each slide, compare sFreq to pFreq (or track a 'matches' counter to avoid a full map comparison each time) and record the start index if they match.

```
m = len(p), n = len(s)
if n < m: return []
pFreq = frequency map of p
sFreq = frequency map of s[0 .. m-1]
result = []
if sFreq == pFreq: result.append(0)
for right from m to n-1:
    left = right - m
    sFreq[s[right]] += 1
    sFreq[s[left]] -= 1
    if sFreq[s[left]] == 0: remove s[left] from sFreq
    if sFreq == pFreq:
        result.append(left + 1)
return result
```

 **Time:** $O(n)$ // amortized $O(1)$ work per slide with a running frequency map  **Space:** $O(m) = O(26)$ in practice for lowercase-letter alphabets

Pattern: Fast & Slow Pointers

How to identify this pattern

- The input is a linked list (or can be modeled as one — e.g. a sequence generated by repeatedly applying a function).
- You need to detect a cycle, find its starting point, or find the middle of a list without knowing its length in advance.
- You cannot randomly access elements by index the way you can in an array, so a two-pointer 'from both ends' approach (like in the Two Pointers pattern) doesn't apply — instead both pointers start at the SAME end and move at different SPEEDS.
- A brute-force solution needs a second pass, extra storage (a hash set of visited nodes), or first computing the length of the list — fast & slow pointers do it in one pass with $O(1)$ space.
- Keywords to watch for: "cycle", "detect loop", "middle of the list", "nth node from the end", "happy number" (sequences that may loop forever).

Q11. Middle of the Linked List

Problem Statement

Given the head of a singly linked list, return the middle node. If there are two middle nodes, return the second middle node.

Example $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow$ Output: node with value 3 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow$
Output: node with value 3

 **Pattern Insight — how to spot it:** If a fast pointer moves 2 steps for every 1 step the slow pointer takes, then when fast reaches the end, slow has covered exactly half the distance — landing it on the middle. This avoids a first pass just to count the list's length.

Brute Force Approach

First pass: count the total number of nodes n . Second pass: walk $n/2$ steps to reach the middle.

```
n = 0, node = head
while node != null:
    n = n + 1
    node = node.next
node = head
for i from 0 to n/2 - 1:
    node = node.next
return node
```

 **Time:** $O(n)$ // two passes, still linear  **Space:** $O(1)$

Optimized Approach

slow and fast both start at head. Each loop iteration, slow moves 1 step and fast moves 2 steps. When fast reaches the end (or null), slow is at the middle.

```
slow = head, fast = head
while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next
return slow
```

 **Time:** $O(n)$ // single pass  **Space:** $O(1)$

Q12. Linked List Cycle

Problem Statement

Given the head of a linked list, determine if the linked list has a cycle in it.

Example 3 → 2 → 0 → -4 → (back to node with value 2) → Output: true



Pattern Insight — how to spot it: This is the canonical fast & slow pointers problem, also called 'Floyd's Cycle Detection' or the 'tortoise and hare' algorithm. If there's a cycle, the fast pointer (moving 2x speed) will eventually lap the slow pointer and they will meet INSIDE the cycle. If there's no cycle, fast simply reaches the end (null) first.

Brute Force Approach

Traverse the list, storing every visited node in a hash set. If you revisit a node already in the set, there's a cycle.

```
visited = empty set
node = head
while node != null:
    if node in visited:
        return true
    visited.add(node)
    node = node.next
return false
```

🕒 **Time:** O(n) 📁 **Space:** O(n) // hash set of visited nodes

Optimized Approach

slow and fast both start at head. Move slow by 1, fast by 2, each iteration. If they ever become equal (same node), there's a cycle. If fast hits null, there's no cycle.

```
slow = head, fast = head
while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next
    if slow == fast:
        return true
return false
```

🕒 **Time:** O(n) 📁 **Space:** O(1) // no extra storage

Q13. Linked List Cycle II (Find the Start of the Cycle)

Problem Statement

Given the head of a linked list, return the node where the cycle begins. If there is no cycle, return null.

Example 3 → 2 → 0 → -4 → (back to node with value 2) → Output: node with value 2

 **Pattern Insight — how to spot it:** This extends Floyd's algorithm with a math trick: once slow and fast meet inside the cycle, resetting one pointer to head and advancing BOTH pointers one step at a time will make them meet exactly at the cycle's start. This works because of the distance relationship between head-to-cycle-start and meeting-point-to-cycle-start — memorize the two-phase recipe, not just the proof.

Brute Force Approach

Traverse with a hash set of visited nodes; the first node you see twice is the cycle's start.

```
visited = empty set
node = head
while node != null:
    if node in visited:
        return node
    visited.add(node)
    node = node.next
return null
```

 **Time:** O(n)  **Space:** O(n)

Optimized Approach

Phase 1: run slow/fast until they meet (as in cycle detection); if fast reaches null, return null (no cycle). Phase 2: reset one pointer to head; move both remaining pointers one step at a time until they meet again — that meeting point is the cycle's start.

```
slow = head, fast = head
while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next
    if slow == fast:
        // Phase 2: find the cycle start
        ptr = head
        while ptr != slow:
            ptr = ptr.next
            slow = slow.next
        return ptr
return null // no cycle
```

 **Time:** O(n)  **Space:** O(1)

Q14. Happy Number

Problem Statement

A happy number is defined by: starting with a positive integer, replace it by the sum of the squares of its digits, repeat until the number equals 1 (happy), or it loops endlessly in a cycle that does not include 1 (not happy). Determine if n is happy.

Example $n = 19 \rightarrow 1^2+9^2=82 \rightarrow 8^2+2^2=68 \rightarrow 6^2+8^2=100 \rightarrow 1^2+0^2+0^2=1 \rightarrow$ Output: true



Pattern Insight — how to spot it: This isn't literally a linked list, but the sequence of 'next number' values forms an implicit chain — exactly like linked-list nodes. Repeatedly applying a deterministic function to a value and needing to detect if you loop forever is the abstract shape of the Fast & Slow Pointers pattern, even with no explicit list structure.

Brute Force Approach

Keep a hash set of all numbers seen so far. If the sum-of-squares process reaches 1, it's happy; if it repeats a previously seen number, it's not.

```
seen = empty set
while n != 1 and n not in seen:
    seen.add(n)
    n = sumOfSquaresOfDigits(n)
return n == 1
```

Time: $O(\log n)$ per transformation, bounded number of transformations before repeat/1 **Space:** $O(k)$ // k = number of distinct values seen before cycling

Optimized Approach

$slow = n$, $fast = \text{sumOfSquaresOfDigits}(n)$. Repeatedly apply the transform: slow moves 1 step, fast moves 2 steps (the transform applied twice). If $slow == fast$ and it's not 1, we've hit a cycle $\rightarrow$ not happy. If either reaches 1, it's happy.

```
slow = n, fast = sumOfSquaresOfDigits(n)
while fast != 1 and slow != fast:
    slow = sumOfSquaresOfDigits(slow)
    fast = sumOfSquaresOfDigits(sumOfSquaresOfDigits(fast))
return fast == 1
```

Time: Same asymptotic bound, but $O(1)$ extra space instead of a growing hash set **Space:** $O(1)$

Q15. Palindrome Linked List

Problem Statement

Given the head of a singly linked list, return true if it is a palindrome.

Example 1 → 2 → 2 → 1 → Output: true 1 → 2 → Output: false

 **Pattern Insight — how to spot it:** Combine fast & slow (to find the middle in one pass) with an in-place reversal of the second half, then compare the two halves with two pointers. This avoids the $O(n)$ extra space of copying the list into an array to check symmetry.

Brute Force Approach

Copy all node values into an array, then use two pointers (or direct reversal) to check if the array reads the same forwards and backwards.

```
values = []
node = head
while node != null:
    values.append(node.val)
    node = node.next
return values == reverse(values)
```

 **Time:** $O(n)$  **Space:** $O(n)$ // array copy of all values

Optimized Approach

1) Use slow/fast pointers to find the middle of the list. 2) Reverse the second half of the list in place. 3) Compare the first half and reversed second half node-by-node with two pointers. 4) (Optionally) reverse the second half back to restore the list.

```
slow = head, fast = head
while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next
secondHalf = reverseList(slow) // in-place reversal
firstHalf = head
isPalindrome = true
p1 = firstHalf, p2 = secondHalf
while p2 != null:
    if p1.val != p2.val:
        isPalindrome = false
        break
    p1 = p1.next
    p2 = p2.next
return isPalindrome
```

 **Time:** $O(n)$  **Space:** $O(1)$ // in-place reversal, no extra array

Q16. Reorder List

Problem Statement

Given the head of a singly linked list $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$, reorder it to: $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$. You may not modify node values, only node links.

Example $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow$ Output: $1 \rightarrow 4 \rightarrow 2 \rightarrow 3$

 **Pattern Insight — how to spot it:** Same three-step recipe as Palindrome Linked List: find the middle with fast & slow, reverse the second half, then MERGE the two halves alternately (instead of just comparing them). Recognizing that several linked-list problems share this 'split, reverse, recombine' skeleton is a big time-saver.

Brute Force Approach

Store all nodes in an array/list (random access), then rebuild the list by alternating from the front and back of the array.

```
nodes = []
node = head
while node != null:
    nodes.append(node)
    node = node.next
i, j = 0, len(nodes) - 1
while i < j:
    nodes[i].next = nodes[j]
    i = i + 1
    if i == j: break
    nodes[j].next = nodes[i]
    j = j - 1
nodes[i].next = null
```

 **Time:** $O(n)$  **Space:** $O(n)$ // array of node references

Optimized Approach

1) Find the middle with slow/fast. 2) Reverse the second half in place. 3) Merge the first half and reversed second half by alternating next pointers.

```
slow = head, fast = head
while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next
second = reverseList(slow)
slow.next = null // cut the first half cleanly, if needed
first = head
while second.next != null:
    firstNext = first.next
    first.next = second
    first = firstNext
    secondNext = second.next
    second.next = first
    second = secondNext
```

 **Time:** $O(n)$  **Space:** $O(1)$

Q17. Circular Array Loop

Problem Statement

Given a circular array 'nums' of non-zero integers where `nums[i]` tells you the number of indices to move forward (positive) or backward (negative) from index `i`, determine if there is a cycle in the array that: has more than one element, moves in a single direction (all forward or all backward), and does not degenerate into a single-element loop.

Example `nums = [2, -1, 1, 2, 2]` → Output: `true` (`0 → 2 → 3 → 0`)

 **Pattern Insight — how to spot it:** Fast & slow pointers work here too, but 'next' is defined by a formula (`index + nums[index]`, wrapped circularly) instead of a `.next` link — a reminder that this pattern applies to ANY implicit sequence, not just literal linked lists. Extra care is needed to enforce 'single direction' and reject trivial one-node self-loops.

Brute Force Approach

From every starting index, simulate stepping through the array (up to `n` steps), checking if you return to the start while respecting the direction rule; mark visited indices per attempt to avoid infinite loops in the simulation itself.

```
for start from 0 to n-1:
    slow = start
    for step from 1 to n:
        slow = nextIndex(nums, slow)
        if slow == start and step > 1:
            return true
    return false
```

 **Time:** $O(n^2)$ // `n` starting points, up to `n` steps each  **Space:** $O(1)$

Optimized Approach

For each index not yet ruled out, run fast & slow pointers using `nextIndex()` as the step function, both constrained to the same direction (sign of `nums[start]`). If `slow == fast` (and it's not a single-node self-loop), a valid cycle exists. Otherwise mark all indices visited this attempt as dead ends (direction changed or self loop) to avoid rework.

```
function nextIndex(nums, i):
    n = len(nums)
    return ((i + nums[i]) % n + n) % n    // handle negative wraparound

for start from 0 to n-1:
    if nums[start] == 0: continue    // already marked dead
    slow = start, fast = start
    direction = nums[start] > 0
    while true:
        slow = nextIndex(nums, slow)
        if directionChanged(nums, slow, direction) or nums[slow] == 0: break
        fast = nextIndex(nums, fast)
        if directionChanged(nums, fast, direction) or nums[fast] == 0: break
        fast = nextIndex(nums, fast)
        if directionChanged(nums, fast, direction) or nums[fast] == 0: break
        if slow == fast:
            if slow == nextIndex(nums, slow): break    // single-element loop, reject
```

```
        return true
    // mark this whole chain as dead (visited, won't form a cycle)
    mark nums[start .. chain] as 0
    return false
```

🕒 **Time:** $O(n)$ // amortized — each index is marked dead at most once 📁 **Space:** $O(1)$ // modifies nums in place as visited markers

Pattern: Cyclic Sort

How to identify this pattern

- You're given an array containing numbers in a known, contiguous range — most commonly $1..n$ or $0..n-1$ — possibly with duplicates or missing values.
- You're asked to find a missing number, a duplicate number, or all numbers that appear more/less than expected.
- The problem hints you can solve it in $O(n)$ time and $O(1)$ extra space (explicitly, or by saying 'without using extra space' / 'in place').
- The brute-force uses sorting ($O(n \log n)$) or a hash set/map ($O(n)$ space) — cyclic sort achieves $O(n)$ time and $O(1)$ space by placing each number at its 'correct' index directly.
- Keywords to watch for: "numbers from 1 to n", "missing number", "duplicate number", "in place", "without extra space".

Q18. Cyclic Sort (the base pattern)

Problem Statement

Given an array 'nums' containing n distinct numbers taken from the range [1, n] (i.e. no duplicates, but not necessarily sorted), sort the array in place using O(1) extra space so that $\text{nums}[i] == i+1$ for every valid i.

Example `nums = [3, 1, 5, 4, 2]` → Output: `[1, 2, 3, 4, 5]`

 **Pattern Insight — how to spot it:** Because the values are exactly 1..n, each value has one 'correct home': value v belongs at index v-1. Walk the array with an index i; if `nums[i]` isn't already at its correct home, swap it there. Repeat at the same i until it IS correct, then move on. Every swap places at least one number in its final position, which is what bounds this to O(n) total swaps.

Brute Force Approach

Just sort the array with a general-purpose sorting algorithm.

```
sort(nums) // using any comparison sort
return nums
```

 **Time:** $O(n \log n)$  **Space:** $O(\log n)$ to $O(n)$ // sort's internal stack/buffer

Optimized Approach

i = 0. While i < n: compute `correctIndex = nums[i] - 1`. If `nums[i]` is not already at `correctIndex`, swap `nums[i]` and `nums[correctIndex]`. Otherwise move i forward. Each swap fixes at least one element permanently.

```
i = 0
while i < n:
    correctIndex = nums[i] - 1
    if nums[i] != nums[correctIndex]:
        swap(nums[i], nums[correctIndex])
    else:
        i = i + 1
return nums
```

 **Time:** $O(n)$ // each swap places a number in its final home; at most n swaps total  **Space:** $O(1)$ // in place

Q19. Find the Missing Number

Problem Statement

Given an array 'nums' containing n distinct numbers taken from the range [0, n] (so exactly one number in that range is missing), find the missing number.

Example nums = [3, 0, 1] → Output: 2 (n=3, range is [0,3], 2 is missing)

 **Pattern Insight — how to spot it:** Range is [0, n] here (n+1 possible values, n slots) — a classic sign that ONE value is always missing after cyclic-sort placement. Run cyclic sort mapping value v to index v (not v-1, since the range starts at 0), then scan for the first index where nums[index] != index — that index is the missing number (or n itself if every slot 0..n-1 is correctly filled).

Brute Force Approach

Sum 0..n using the formula $n(n+1)/2$, subtract the actual sum of nums — the difference is the missing number.

```
expectedSum = n * (n + 1) / 2
actualSum = sum(nums)
return expectedSum - actualSum
```

 **Time:** $O(n)$  **Space:** $O(1)$ // this arithmetic trick is actually optimal too, but doesn't generalize to duplicates/multiple-missing variants the way cyclic sort does

Optimized Approach

Cyclic sort: for each i, if nums[i] < n and nums[i] != i, swap it to index nums[i]. After sorting, scan for the first i where nums[i] != i — that i is the answer. If none found, the missing number is n.

```
i = 0
while i < n:
    correctIndex = nums[i]
    if nums[i] < n and nums[i] != nums[correctIndex]:
        swap(nums[i], nums[correctIndex])
    else:
        i = i + 1
for i from 0 to n-1:
    if nums[i] != i:
        return i
return n
```

 **Time:** $O(n)$  **Space:** $O(1)$

Q20. Find All Missing Numbers

Problem Statement

Given an array 'nums' of n integers where $\text{nums}[i]$ is in the range $[1, n]$, return an array of all the integers in $[1, n]$ that do not appear in nums.

Example `nums = [4,3,2,7,8,2,3,1]` → Output: `[5, 6]`

 **Pattern Insight — how to spot it:** Same cyclic-sort placement as the base pattern (value v belongs at index $v-1$), but this time duplicates mean not every slot can be filled — some values will collide and never find a home. After sorting, ANY index i where $\text{nums}[i] \neq i+1$ represents a missing value ($i+1$).

Brute Force Approach

Use a hash set of all numbers present, then check which numbers in $[1, n]$ are absent.

```
present = set(nums)
result = []
for v from 1 to n:
    if v not in present:
        result.append(v)
return result
```

 **Time:** $O(n)$  **Space:** $O(n)$ // hash set

Optimized Approach

Cyclic sort in place (value v goes to index $v-1$, skipping if already correct or if the target already holds that value — duplicates just get left in place). Then scan: any index i where $\text{nums}[i] \neq i+1$ means ($i+1$) is missing.

```
i = 0
while i < n:
    correctIndex = nums[i] - 1
    if nums[i] != nums[correctIndex]:
        swap(nums[i], nums[correctIndex])
    else:
        i = i + 1
result = []
for i from 0 to n-1:
    if nums[i] != i + 1:
        result.append(i + 1)
return result
```

 **Time:** $O(n)$  **Space:** $O(1)$ // excluding the output array

Q21. Find the Duplicate Number

Problem Statement

Given an array 'nums' containing $n+1$ integers where each integer is in the range $[1, n]$, and there is exactly one repeated number (possibly repeated more than once), find that duplicate — without modifying the array and using $O(1)$ extra space.

Example nums = [1,3,4,2,2] → Output: 2

 **Pattern Insight — how to spot it:** The 'without modifying the array' constraint rules out plain cyclic sort (which swaps in place). But the setup — $n+1$ numbers in range $[1, n]$ — is exactly the shape that makes the array behave like a linked list with a cycle: treat index i as pointing to index $\text{nums}[i]$. The duplicate value creates two indices pointing to the same place, forming a cycle, so Floyd's fast & slow pointer cycle detection finds it in $O(1)$ space without mutating anything.

Brute Force Approach

Use a hash set: the first number you see already in the set is the duplicate.

```
seen = empty set
for num in nums:
    if num in seen:
        return num
    seen.add(num)
```

 **Time:** $O(n)$  **Space:** $O(n)$ // hash set (violates the $O(1)$ space requirement, but a fine warm-up)

Optimized Approach

Treat nums as a function $f(i) = \text{nums}[i]$, forming an implicit linked list starting at index 0. Because two different indices map to the duplicate value, this structure has a cycle. Run Floyd's algorithm (same two phases as Linked List Cycle II) to find the cycle's entry point — that value is the duplicate.

```
slow = nums[0], fast = nums[0]
do:
    slow = nums[slow]
    fast = nums[nums[fast]]
while slow != fast

ptr = nums[0]
while ptr != slow:
    ptr = nums[ptr]
    slow = nums[slow]
return ptr
```

 **Time:** $O(n)$  **Space:** $O(1)$ // no extra structures, array left unmodified

Q22. Find All Duplicates in an Array

Problem Statement

Given an array 'nums' of n integers where each integer is in the range [1, n], and each integer appears once or twice, return an array of all the integers that appear twice.

Example nums = [4,3,2,7,8,2,3,1] → Output: [2, 3]

 **Pattern Insight — how to spot it:** Same placement idea as Find All Missing Numbers — value v belongs at index v-1 — but this time you read off the DUPLICATES instead of the gaps: after cyclic sort, any index i where nums[i] != i+1 means the number currently sitting there (nums[i]) is a duplicate of the number that should have been there.

Brute Force Approach

Track frequency of each number with a hash map; collect numbers with frequency 2.

```
freq = empty map
for num in nums:
    freq[num] += 1
result = []
for num, count in freq:
    if count == 2:
        result.append(num)
return result
```

 **Time:** O(n)  **Space:** O(n)

Optimized Approach

Cyclic sort by placing each value at index value-1 where possible. Then scan: wherever nums[i] != i+1, the value nums[i] is a duplicate (its 'correct' slot i.e. index nums[i]-1 is already occupied by itself).

```
i = 0
while i < n:
    correctIndex = nums[i] - 1
    if nums[i] != nums[correctIndex]:
        swap(nums[i], nums[correctIndex])
    else:
        i = i + 1
result = []
for i from 0 to n-1:
    if nums[i] != i + 1:
        result.append(nums[i])
return result
```

 **Time:** O(n)  **Space:** O(1) // excluding output array

Q23. Set Mismatch

Problem Statement

You have a set of integers originally containing all numbers from 1 to n. Unfortunately, due to an error, one number got duplicated (replacing another number), leaving that other number missing. Given the resulting array 'nums', return [duplicate, missing].

Example nums = [1,2,2,4] → Output: [2, 3]



Pattern Insight — how to spot it: This is Find the Duplicate and Find the Missing Number solved together in one cyclic-sort pass — a great example of how the same placement routine answers multiple related questions depending on what you check afterward.

Brute Force Approach

Count frequency of every number 1..n using a hash map/array; the number with frequency 2 is the duplicate, the number with frequency 0 is missing.

```
freq = array of size n+1, all 0
for num in nums:
    freq[num] += 1
duplicate, missing = -1, -1
for v from 1 to n:
    if freq[v] == 2: duplicate = v
    if freq[v] == 0: missing = v
return [duplicate, missing]
```

⌚ **Time:** O(n) 📁 **Space:** O(n) // frequency array

Optimized Approach

Cyclic sort (value v to index v-1). Then scan for the index i where nums[i] != i+1: nums[i] is the duplicate, and (i+1) is the missing number.

```
i = 0
while i < n:
    correctIndex = nums[i] - 1
    if nums[i] != nums[correctIndex]:
        swap(nums[i], nums[correctIndex])
    else:
        i = i + 1
for i from 0 to n-1:
    if nums[i] != i + 1:
        return [nums[i], i + 1] // [duplicate, missing]
```

⌚ **Time:** O(n) 📁 **Space:** O(1)

Q24. First Missing Positive

Problem Statement

Given an unsorted integer array 'nums' (which may include negatives, zero, and numbers larger than n), find the smallest missing positive integer, in $O(n)$ time and $O(1)$ extra space.

Example nums = [3,4,-1,1] → Output: 2 nums = [1,2,0] → Output: 3

 **Pattern Insight — how to spot it:** The answer is always somewhere in $[1, n+1]$ — you can't have all of $1..n$ missing AND every extra slot filled with something unhelpful, so you never need to look beyond $n+1$. This lets you apply cyclic sort even though the input isn't restricted to $[1, n]$ up front: just IGNORE any value that's out of range (≤ 0 or $> n$) during placement, since it can never be the answer.

Brute Force Approach

Put all positive numbers into a hash set, then check 1, 2, 3, ... in order until one is missing.

```
present = set of positive numbers in nums
candidate = 1
while candidate in present:
    candidate = candidate + 1
return candidate
```

 **Time:** $O(n)$  **Space:** $O(n)$ // hash set

Optimized Approach

Cyclic sort, but only place values v where $1 \leq v \leq n$ (skip/ignore anything outside that range — it can never be the answer since the answer is at most $n+1$). After sorting, the first index i where $\text{nums}[i] \neq i+1$ gives the answer $i+1$; if every slot is correct, the answer is $n+1$.

```
n = len(nums)
i = 0
while i < n:
    correctIndex = nums[i] - 1
    if nums[i] > 0 and nums[i] <= n and nums[i] != nums[correctIndex]:
        swap(nums[i], nums[correctIndex])
    else:
        i = i + 1
for i from 0 to n-1:
    if nums[i] != i + 1:
        return i + 1
return n + 1
```

 **Time:** $O(n)$  **Space:** $O(1)$

Pattern: Merge Intervals

How to identify this pattern

- The input is a collection of intervals, each with a start and an end (times, ranges, positions).
- You need to combine overlapping intervals, insert a new interval into an existing sorted set, find intersections between two interval lists, or determine the minimum resources (rooms, arrows, etc.) needed to handle all intervals.
- Two intervals $[a, b]$ and $[c, d]$ overlap exactly when $a \leq d$ and $c \leq b$ (or equivalently, when they are sorted by start: the next interval's start $\leq$ current interval's end).
- The brute force compares every pair of intervals ($O(n^2)$); sorting by start time first turns this into a single linear sweep ($O(n \log n)$ dominated by the sort).
- Keywords to watch for: "intervals", "meetings/appointments", "schedule", "overlapping", "merge", "free time".

Q25. Merge Intervals

Problem Statement

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example `intervals = [[1,3],[2,6],[8,10],[15,18]]` → Output: `[[1,6],[8,10],[15,18]]`

 **Pattern Insight — how to spot it:** Sort by start time first. Once sorted, you only ever need to compare each interval to the LAST interval you've already merged (not all previous ones) — if the current interval's start is $\leq$ the last merged interval's end, they overlap and you extend the end; otherwise you start a new merged interval. Sorting is what turns an $O(n^2)$ pairwise comparison into a single $O(n)$ sweep.

Brute Force Approach

Repeatedly scan all pairs of intervals; whenever two overlap, merge them into one and restart the scan, until no pair overlaps.

```
merged = copy of intervals
changed = true
while changed:
    changed = false
    for i from 0 to len(merged)-1:
        for j from i+1 to len(merged)-1:
            if overlaps(merged[i], merged[j]):
                merged[i] = union(merged[i], merged[j])
                remove merged[j]
                changed = true
                break out of both loops
return merged
```

 **Time:** $O(n^3)$ // repeated $O(n^2)$ passes  **Space:** $O(n)$

Optimized Approach

Sort intervals by start. Initialize result with the first interval. For each subsequent interval, if it overlaps the last interval in result ($\text{current.start} \leq \text{last.end}$), extend $\text{last.end} = \max(\text{last.end}, \text{current.end})$; otherwise append current as a new interval.

```
sort intervals by start
result = [intervals[0]]
for i from 1 to n-1:
    last = result[-1]
    current = intervals[i]
    if current.start <= last.end:
        last.end = max(last.end, current.end)
    else:
        result.append(current)
return result
```

 **Time:** $O(n \log n)$ // dominated by sorting  **Space:** $O(n)$ // for sorting + output ($O(\log n)$ to $O(n)$ extra if in-place output not counted)

Q26. Insert Interval

Problem Statement

Given a set of non-overlapping intervals sorted by start time, and a new interval, insert the new interval into the correct position and merge if necessary. Return the resulting list of intervals.

Example intervals = [[1,3],[6,9]], newInterval = [2,5] → Output: [[1,5],[6,9]]

 **Pattern Insight — how to spot it:** Because the input is ALREADY sorted and non-overlapping, you don't need to re-sort anything — just walk through in three phases: (1) copy all intervals that end before the new interval starts (no overlap, come first), (2) merge all intervals that overlap the new interval by expanding its bounds, (3) copy all intervals that start after the merged interval ends. This is the 'Merge Intervals' logic specialized to a single insertion, done in one linear pass.

Brute Force Approach

Append the new interval to the list, then run the general Merge Intervals algorithm (sort + sweep) on the whole thing.

```
all = intervals + [newInterval]
sort all by start
return mergeIntervals(all) // same sweep as the Merge Intervals problem
```

 **Time:** $O(n \log n)$ // re-sorting even though input was already sorted  **Space:** $O(n)$

Optimized Approach

Phase 1: add all intervals ending before newInterval starts. Phase 2: merge all intervals that overlap newInterval into it (expanding newInterval's start/end). Phase 3: add newInterval, then add all remaining intervals.

```
result = []
i = 0, n = len(intervals)
while i < n and intervals[i].end < newInterval.start:
    result.append(intervals[i])
    i = i + 1
while i < n and intervals[i].start <= newInterval.end:
    newInterval.start = min(newInterval.start, intervals[i].start)
    newInterval.end = max(newInterval.end, intervals[i].end)
    i = i + 1
result.append(newInterval)
while i < n:
    result.append(intervals[i])
    i = i + 1
return result
```

 **Time:** $O(n)$ // single linear pass, no sorting needed  **Space:** $O(n)$ // for the output

Q27. Non-overlapping Intervals (Erase Overlap Intervals)

Problem Statement

Given an array of intervals, find the minimum number of intervals you need to remove so that the rest of the intervals are non-overlapping.

Example intervals = [[1,2],[2,3],[3,4],[1,3]] → Output: 1 (remove [1,3])

 **Pattern Insight — how to spot it:** This is a classic greedy-on-intervals problem: sort by END time (not start!). Then greedily keep an interval if its start is $\geq$ the end of the last kept interval; otherwise it overlaps, so you must remove one — and greedily removing the one with the LATER end time (i.e., discard the current one, keep the previous 'last') always leaves the most room for future intervals. Sorting by end time is the key trick that makes the greedy choice provably optimal.

Brute Force Approach

Try all subsets of intervals to remove, check if what remains is non-overlapping, and take the minimum removal count (exponential, for illustration only).

```
best = n
for each subset S of intervals to keep:
    if S is pairwise non-overlapping:
        best = min(best, n - len(S))
return best
```

 **Time:** $O(2^n)$ // exponential subset search  **Space:** $O(n)$

Optimized Approach

Sort intervals by end time. Track lastEnd = -infinity and a removal count. For each interval in order: if its start < lastEnd, it overlaps the last kept interval — remove it (increment count), and DON'T update lastEnd (keep the smaller end). Otherwise keep it and set lastEnd = its end.

```
sort intervals by end
lastEnd = -infinity
removals = 0
for interval in intervals:
    if interval.start < lastEnd:
        removals = removals + 1 // discard this interval
    else:
        lastEnd = interval.end
return removals
```

 **Time:** $O(n \log n)$ // dominated by sorting  **Space:** $O(1)$ to $O(n)$ depending on sort implementation

Q28. Meeting Rooms II

Problem Statement

Given an array of meeting time intervals, return the minimum number of conference rooms required to hold all the meetings without conflict.

Example intervals = [[0,30],[5,10],[15,20]] → Output: 2

 **Pattern Insight — how to spot it:** This is 'maximum number of intervals overlapping at the same time' — think of it as a min-heap of currently-occupied rooms' end times, or equivalently a chronological sweep of +1 at every start and -1 at every end. Sort starts and ends SEPARATELY; the running count of (starts processed - ends processed) at any point tells you exactly how many rooms are in use, and the peak of that count is the answer.

Brute Force Approach

For each meeting, check against every other meeting to build an overlap graph, then estimate rooms needed by simulation — inefficient and error-prone; shown here as the naive baseline of assigning rooms one meeting at a time by scanning all existing rooms for a free one.

```
rooms = [] // each room stores its current end time
for meeting in sort(intervals by start):
    assigned = false
    for room in rooms:
        if room.endTime <= meeting.start:
            room.endTime = meeting.end
            assigned = true
            break
    if not assigned:
        rooms.append(new room with endTime = meeting.end)
return len(rooms)
```

 **Time:** $O(n^2)$ // scanning all existing rooms for each meeting  **Space:** $O(n)$

Optimized Approach

Split into a 'starts' array and an 'ends' array, both sorted independently. Use two pointers walking through both arrays in time order: whenever a start occurs before the earliest current end, a new room is needed (increment rooms in use); whenever an end occurs first, a room frees up (decrement). Track the maximum rooms in use at any point.

```
starts = sort([i.start for i in intervals])
ends = sort([i.end for i in intervals])
s = 0, e = 0
roomsInUse = 0, maxRooms = 0
while s < n:
    if starts[s] < ends[e]:
        roomsInUse = roomsInUse + 1
        s = s + 1
        maxRooms = max(maxRooms, roomsInUse)
    else:
        roomsInUse = roomsInUse - 1
        e = e + 1
return maxRooms
```

 **Time:** $O(n \log n)$ // dominated by sorting the two arrays  **Space:** $O(n)$ // starts and ends arrays

Q29. Interval List Intersections

Problem Statement

Given two lists of closed intervals, each sorted in ascending order by start time and each list of pairwise disjoint intervals, return the intersection of these two interval lists.

Example A = [[0,2],[5,10],[13,23]], B = [[1,5],[8,12]] → Output: [[1,2],[5,5],[8,10]]

 **Pattern Insight — how to spot it:** Both lists are independently sorted and disjoint — the classic setup for a two-pointer merge (like merging two sorted arrays). At each step, the overlap of the current interval from A and the current interval from B (if any) is [max(starts), min(ends)]; then you always advance the pointer belonging to whichever interval ends first, since that interval can't overlap anything further.

Brute Force Approach

Compare every interval in A against every interval in B, computing and collecting any overlap.

```
result = []
for a in A:
    for b in B:
        lo = max(a.start, b.start)
        hi = min(a.end, b.end)
        if lo <= hi:
            result.append([lo, hi])
return result
```

 **Time:** $O(n * m)$ // $n = \text{len}(A)$, $m = \text{len}(B)$  **Space:** $O(n*m)$ worst case for output, though typically much smaller

Optimized Approach

$i = 0$ (pointer into A), $j = 0$ (pointer into B). Compute overlap of $A[i]$ and $B[j]$; if it's valid ($lo \leq hi$), record it. Then advance whichever of $A[i]$ or $B[j]$ has the SMALLER end (it can no longer overlap anything else).

```
i = 0, j = 0
result = []
while i < len(A) and j < len(B):
    lo = max(A[i].start, B[j].start)
    hi = min(A[i].end, B[j].end)
    if lo <= hi:
        result.append([lo, hi])
    if A[i].end < B[j].end:
        i = i + 1
    else:
        j = j + 1
return result
```

 **Time:** $O(n + m)$ // single linear merge pass  **Space:** $O(n + m)$ // for the output in the worst case

Q30. Minimum Number of Arrows to Burst Balloons

Problem Statement

There are balloons represented as intervals $[xstart, xend]$ along a horizontal line. An arrow shot at position x bursts every balloon whose interval contains x . Find the minimum number of arrows needed to burst all balloons.

Example `points = [[10,16],[2,8],[1,6],[7,12]]` → Output: 2

 **Pattern Insight — how to spot it:** This is 'Non-overlapping Intervals' in disguise: an arrow can burst every balloon in one overlapping cluster at once, so the answer equals the number of non-overlapping (in the sense of NOT sharing any point) groups. Sort by end position, greedily shoot an arrow at the end of the first balloon, and skip every subsequent balloon whose start is $\leq$ that arrow's position (they're all popped); when a balloon starts after it, you need a new arrow.

Brute Force Approach

Try every candidate arrow position (or every subset of balloons) and simulate which are burst — exponential/very slow, shown only as the naive baseline of a greedy-but-unsorted scan that doesn't guarantee minimality.

```
// Naive: without sorting, greedily grouping balloons doesn't guarantee
// the minimum number of arrows – you'd need to try many groupings.
// (Left as exponential search over subsets for illustration.)
```

 **Time:** Exponential without sorting  **Space:** $O(n)$

Optimized Approach

Sort balloons by `xend`. `arrowPos = points[0].end`, `arrows = 1`. For each subsequent balloon, if its start $>$ `arrowPos`, it needs a new arrow: increment `arrows` and set `arrowPos = this balloon's end`. Otherwise it's already burst by the current arrow.

```
sort points by xend
arrows = 1
arrowPos = points[0].end
for i from 1 to n-1:
    if points[i].start > arrowPos:
        arrows = arrows + 1
        arrowPos = points[i].end
return arrows
```

 **Time:** $O(n \log n)$ // dominated by sorting  **Space:** $O(1)$ to $O(n)$ depending on sort implementation

Q31. Employee Free Time

Problem Statement

Given a list of schedules, one per employee, where each schedule is a list of non-overlapping intervals sorted by start time, return the list of finite intervals representing common, positive-length free time shared by ALL employees, sorted in order.

Example `schedules = [[[1,2],[5,6]], [[1,3]], [[4,10]]]` → Output: `[[3,4]]`

 **Pattern Insight — how to spot it:** Flatten every employee's intervals into one big list, then it's exactly the Merge Intervals problem — merge all busy intervals across everyone, and the GAPS between consecutive merged intervals are the company-wide free time. This shows how a 'multi-list' interval problem often reduces to the single-list version after one flattening step.

Brute Force Approach

For every possible time point across the whole range, check whether ANY employee is busy at that point — extremely coarse and not truly $O(n \log n)$, shown as a conceptual (impractical) baseline.

```
// Conceptual only: scanning every unit of time and checking all
// schedules for a conflict is far too slow / not well-defined
// for continuous or large ranges.
```

 **Time:** Impractically large / ill-defined  **Space:** $O(\text{range})$

Optimized Approach

Flatten all employees' intervals into one list, sort by start. Merge overlapping intervals as in the standard Merge Intervals sweep. Then walk through the merged list: whenever `merged[i].end < merged[i+1].start`, that gap `[merged[i].end, merged[i+1].start]` is a free-time interval.

```
all = flatten(schedules) // one list of all intervals across employees
sort all by start
merged = mergeIntervals(all) // same sweep as the Merge Intervals problem
freeTime = []
for i from 0 to len(merged)-2:
    if merged[i].end < merged[i+1].start:
        freeTime.append([merged[i].end, merged[i+1].start])
return freeTime
```

 **Time:** $O(n \log n)$ // n = total intervals across all employees; dominated by sorting  **Space:** $O(n)$

Q32. My Calendar I

Problem Statement

Implement a MyCalendar class that `book(start, end)` adds a new event to the calendar if it does not overlap with any existing event (double booking not allowed), returning `true` if successfully added, `false` otherwise.

Example `book(10,20)→true`, `book(15,25)→false` (overlaps `[10,20]`), `book(20,30)→true` (touching, not overlapping)

 **Pattern Insight — how to spot it:** Unlike the batch problems above, this is an ONLINE / incremental version: events arrive one at a time and you must answer immediately, so you can't just sort everything upfront. Keep the booked events in a sorted structure (a balanced BST / sorted list) so each new booking only needs to check its immediate neighbors (the events right before and after its start) for overlap — the interval-overlap CHECK is the same math as Merge Intervals, just applied incrementally with an efficient lookup structure.

Brute Force Approach

Keep a simple list of booked events; for each new booking, scan the entire list to check for overlap with every existing event.

```
bookings = []
function book(start, end):
    for (s, e) in bookings:
        if start < e and s < end:    // overlap check
            return false
    bookings.append((start, end))
    return true
```

 **Time:** $O(n)$ per booking $\rightarrow O(n^2)$ for n total bookings  **Space:** $O(n)$

Optimized Approach

Store bookings in a sorted map/balanced BST keyed by start time. For a new `(start, end)`, find the booking immediately before start (its end must be $\leq$ start) and the booking immediately after (its start must be $\geq$ end). If both checks pass, insert; otherwise reject.

```
bookings = sorted map (balanced BST) keyed by start time
function book(start, end):
    prevEntry = greatest entry with key <= start
    nextEntry = smallest entry with key >= start
    if prevEntry exists and prevEntry.end > start: return false
    if nextEntry exists and nextEntry.start < end: return false
    insert (start -> end) into bookings
    return true
```

 **Time:** $O(\log n)$ per booking // balanced BST insert/lookup  **Space:** $O(n)$ // stored bookings

Pattern: Modified Binary Search

How to identify this pattern

- The input is sorted, or 'almost sorted' (e.g. rotated), or the SEARCH SPACE is a range of possible answers rather than the array itself.
- You need to find a specific element, a boundary (first/last occurrence, insertion point), or a peak — anything where the search space can be cut roughly in half at every step based on a comparison.
- For 'binary search on the answer' problems, the giveaway is: the answer lies in a range of numbers, and there's a monotonic yes/no condition — as the candidate answer increases, the condition flips from false to true (or true to false) exactly once.
- The brute force scans linearly ($O(n)$) or tries every possible answer value ($O(\text{range})$ or worse) — binary search brings this down to $O(\log n)$ or $O(\log \text{range})$.
- Keywords to watch for: "sorted array", "rotated sorted array", "find minimum/maximum such that...", "smallest/largest value satisfying...", "peak element".

Q33. Binary Search (the base pattern)

Problem Statement

Given a sorted array of distinct integers 'nums' and a target value, return the index of target if it exists, otherwise return -1.

Example nums = [-1,0,3,5,9,12], target = 9 → Output: 4

 **Pattern Insight — how to spot it:** The foundation: maintain [left, right] as the current search space. Look at the middle element — if it's the target, done. If the target is smaller, the target (if present) must be in the LEFT half, so discard the right half (and vice versa). Each comparison halves the search space, giving $O(\log n)$.

Brute Force Approach

Scan every element left to right until the target is found.

```
for i from 0 to n-1:
    if nums[i] == target:
        return i
return -1
```

 **Time:** $O(n)$  **Space:** $O(1)$

Optimized Approach

left = 0, right = n-1. While left <= right: compute mid, compare nums[mid] to target, and narrow the range to the half that could still contain target.

```
left = 0, right = n - 1
while left <= right:
    mid = left + (right - left) / 2
    if nums[mid] == target:
        return mid
    else if nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1
return -1
```

 **Time:** $O(\log n)$  **Space:** $O(1)$

Q34. Find First and Last Position of Element in Sorted Array

Problem Statement

Given a sorted array of integers 'nums' and a target, find the starting and ending position of target. Return [-1, -1] if target is not found. Must run in $O(\log n)$.

Example nums = [5,7,7,8,8,10], target = 8 → Output: [3, 4]

 **Pattern Insight — how to spot it:** A single binary search only finds 'a' match, not necessarily the first or last. Run TWO separate binary searches with a small twist: to find the leftmost occurrence, when you find a match, don't stop — keep searching the LEFT half for an even earlier match. Symmetric logic (search the right half after a match) finds the rightmost occurrence.

Brute Force Approach

Scan the whole array once, tracking the first and last index where `nums[i] == target`.

```
first, last = -1, -1
for i from 0 to n-1:
    if nums[i] == target:
        if first == -1: first = i
        last = i
return [first, last]
```

 **Time:** $O(n)$  **Space:** $O(1)$

Optimized Approach

`findBound(nums, target, findFirst)`: standard binary search, but on a match, instead of returning immediately, record the index and continue narrowing toward the left (if `findFirst`) or right (if not) to squeeze out an earlier/later match. Call it twice.

```
function findBound(nums, target, findFirst):
    left, right = 0, n - 1
    result = -1
    while left <= right:
        mid = left + (right - left) / 2
        if nums[mid] == target:
            result = mid
            if findFirst: right = mid - 1 // keep searching left
            else: left = mid + 1 // keep searching right
        else if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return result

return [findBound(nums, target, true), findBound(nums, target, false)]
```

 **Time:** $O(\log n)$ // two binary searches  **Space:** $O(1)$

Q35. Search in Rotated Sorted Array

Problem Statement

Given a sorted array 'nums' that has been rotated at an unknown pivot, and a target, return the index of target if it exists, else -1. Must run in $O(\log n)$.

Example nums = [4,5,6,7,0,1,2], target = 0 → Output: 4

 **Pattern Insight — how to spot it:** Even after rotation, at least ONE half of any [left, right] window (split at mid) is still normally sorted. Identify which half is sorted by comparing nums[left] to nums[mid]; then check if target falls within that sorted half's range — if yes, recurse/iterate there, if no, go to the other half. This 'figure out which half is sorted, then decide' check is the core trick for every rotated-array problem.

Brute Force Approach

Scan the whole array linearly for target.

```
for i from 0 to n-1:
    if nums[i] == target:
        return i
return -1
```

 **Time:** $O(n)$  **Space:** $O(1)$

Optimized Approach

Standard binary search loop, but at each mid, first determine whether the LEFT half [left..mid] or RIGHT half [mid..right] is sorted (compare nums[left] and nums[mid]). Then check if target lies within the sorted half's value range to decide which side to keep.

```
left = 0, right = n - 1
while left <= right:
    mid = left + (right - left) / 2
    if nums[mid] == target: return mid
    if nums[left] <= nums[mid]: // left half is sorted
        if nums[left] <= target < nums[mid]:
            right = mid - 1
        else:
            left = mid + 1
    else: // right half is sorted
        if nums[mid] < target <= nums[right]:
            left = mid + 1
        else:
            right = mid - 1
return -1
```

 **Time:** $O(\log n)$  **Space:** $O(1)$

Q36. Find Minimum in Rotated Sorted Array

Problem Statement

Given a sorted array 'nums' (of unique elements) rotated at an unknown pivot, find the minimum element. Must run in $O(\log n)$.

Example `nums = [4,5,6,7,0,1,2] → Output: 0` `nums = [11,13,15,17] → Output: 11`

 **Pattern Insight — how to spot it:** The minimum is the one 'discontinuity' point — the only place where an element is smaller than its predecessor. Compare `nums[mid]` to `nums[right]`: if `nums[mid] > nums[right]`, the minimum must be in the right half (past mid); otherwise the minimum is at mid or in the left half (never discard mid itself, since it could BE the minimum) — this careful `right = mid` (not `mid-1`) is what makes the loop correct.

Brute Force Approach

Scan the whole array to find the smallest element.

```
minVal = nums[0]
for i from 1 to n-1:
    minVal = min(minVal, nums[i])
return minVal
```

 **Time:** $O(n)$  **Space:** $O(1)$

Optimized Approach

`left = 0`, `right = n-1`. While `left < right`: compare `nums[mid]` to `nums[right]`. If `nums[mid] > nums[right]`, minimum is strictly to the right of mid → `left = mid + 1`. Otherwise minimum is at mid or to its left → `right = mid` (don't exclude mid). Loop ends when `left == right`, which is the index of the minimum.

```
left = 0, right = n - 1
while left < right:
    mid = left + (right - left) / 2
    if nums[mid] > nums[right]:
        left = mid + 1
    else:
        right = mid
return nums[left]
```

 **Time:** $O(\log n)$  **Space:** $O(1)$

Q37. Search a 2D Matrix

Problem Statement

Given an $m \times n$ matrix where each row is sorted left to right and the first integer of each row is greater than the last integer of the previous row (so the whole matrix is sorted if read row by row), determine if target exists. Must run in $O(\log(m*n))$.

Example matrix = `[[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, target = 3 → Output: true

 **Pattern Insight — how to spot it:** Because the matrix is fully sorted when 'flattened' row by row, you can treat it as ONE sorted 1D array of length $m*n$ and run standard binary search, converting a flat index back to (row, col) with $\text{row} = \text{index} / n$ and $\text{col} = \text{index} \% n$. Recognizing when a 2D structure secretly behaves like a 1D sorted array is the key insight.

Brute Force Approach

Scan every cell of the matrix looking for target.

```
for r from 0 to m-1:
    for c from 0 to n-1:
        if matrix[r][c] == target:
            return true
return false
```

 **Time:** $O(m * n)$  **Space:** $O(1)$

Optimized Approach

Binary search over the virtual flat index range $[0, m*n - 1]$. At each mid, convert to (row, col) and compare `matrix[row][col]` to target, narrowing the range exactly like standard binary search.

```
left = 0, right = m * n - 1
while left <= right:
    mid = left + (right - left) / 2
    row = mid / n, col = mid % n
    val = matrix[row][col]
    if val == target: return true
    else if val < target: left = mid + 1
    else: right = mid - 1
return false
```

 **Time:** $O(\log(m * n))$  **Space:** $O(1)$

Q38. Find Peak Element

Problem Statement

A peak element is one strictly greater than its neighbors. Given an array 'nums' where $\text{nums}[-1]$ and $\text{nums}[n]$ are considered $-\infty$, find any peak element's index. Must run in $O(\log n)$.

Example $\text{nums} = [1, 2, 3, 1] \rightarrow \text{Output: } 2$ ($\text{nums}[2]=3$ is a peak) $\text{nums} = [1, 2, 1, 3, 5, 6, 4] \rightarrow \text{Output: } 1 \text{ or } 5$

 **Pattern Insight — how to spot it:** There's no requirement for the array to be sorted, yet binary search still works because of a local-slope argument: if $\text{nums}[\text{mid}] < \text{nums}[\text{mid}+1]$, the array is 'still rising' at mid, so a peak MUST exist somewhere to the right (worst case, the very last element, bounded by $-\infty$ beyond it). Symmetric logic applies to the left. This is 'binary search on structure/slope', not on sorted order — a very different flavor worth recognizing.

Brute Force Approach

Scan linearly, checking each element against both neighbors.

```
for i from 0 to n-1:
    leftOK = (i == 0) or (nums[i] > nums[i-1])
    rightOK = (i == n-1) or (nums[i] > nums[i+1])
    if leftOK and rightOK:
        return i
```

 **Time:** $O(n)$  **Space:** $O(1)$

Optimized Approach

$\text{left} = 0$, $\text{right} = n-1$. While $\text{left} < \text{right}$: if $\text{nums}[\text{mid}] < \text{nums}[\text{mid}+1]$, a peak lies to the right $\rightarrow \text{left} = \text{mid} + 1$. Otherwise a peak lies at mid or to the left $\rightarrow \text{right} = \text{mid}$. Loop ends at a peak index.

```
left = 0, right = n - 1
while left < right:
    mid = left + (right - left) / 2
    if nums[mid] < nums[mid + 1]:
        left = mid + 1
    else:
        right = mid
return left
```

 **Time:** $O(\log n)$  **Space:** $O(1)$

Q39. Koko Eating Bananas (Binary Search on the Answer)

Problem Statement

Koko has n piles of bananas; pile i has $\text{piles}[i]$ bananas. She can eat at most k bananas per hour from a single pile (if a pile has fewer than k , she finishes it and stops that hour). Given h hours total, find the minimum integer eating speed k such that she can eat all bananas within h hours.

Example `piles = [3,6,7,11]`, `h = 8` → Output: 4

 **Pattern Insight — how to spot it:** This is the archetype of 'binary search on the answer'. The actual array isn't what you binary search over — the ANSWER SPACE (possible values of k , from 1 to $\max(\text{piles})$) is. The condition 'can Koko finish within h hours at speed k ' is monotonic: if speed k works, any larger speed also works (finishes at least as fast); if k fails, any smaller speed also fails. That monotonicity is exactly what a binary search needs — search for the smallest k where the condition flips from false to true.

Brute Force Approach

Try every possible speed k from 1 upward, computing hours needed at that speed, and return the first k that fits within h hours.

```
for k from 1 to max(piles):
    hours = sum(ceil(pile / k) for pile in piles)
    if hours <= h:
        return k
```

 **Time:** $O(\max\text{Pile} * n)$ // trying every speed, summing over all piles each time  **Space:** $O(1)$

Optimized Approach

Binary search k in $[1, \max(\text{piles})]$. For each candidate mid speed, compute total hours needed = $\sum(\text{ceil}(\text{pile} / \text{mid}))$. If $\text{hours} \leq h$, mid works — try smaller ($\text{right} = \text{mid}$). Otherwise mid is too slow — try larger ($\text{left} = \text{mid} + 1$). Converge to the minimum working speed.

```
left = 1, right = max(piles)
while left < right:
    mid = left + (right - left) / 2
    hours = sum(ceil(pile / mid) for pile in piles)
    if hours <= h:
        right = mid          // mid works, try to do better (slower speed)
    else:
        left = mid + 1       // mid too slow, need faster
return left
```

 **Time:** $O(n * \log(\max\text{Pile}))$ // binary search over speed, $O(n)$ work to check each candidate  **Space:** $O(1)$

Q40. Capacity To Ship Packages Within D Days (Binary Search on the Answer)

Problem Statement

A conveyor belt has weights[i] as the weight of the ith package, loaded in order. Ship all packages within D days by choosing the ship's minimum weight capacity, where each day you load packages in order without exceeding capacity. Find the least capacity that ships everything within D days.

Example weights = [1,2,3,4,5,6,7,8,9,10], D = 5 → Output: 15



Pattern Insight — how to spot it: Another binary-search-on-the-answer problem: the search space is the possible ship CAPACITY, from max(weights) (must fit the heaviest single package) to sum(weights) (ship everything in one day). The condition 'can we ship within D days at capacity c' is monotonic — larger capacity never needs MORE days — so binary search for the smallest capacity where the days-needed condition first becomes ≤ D.

Brute Force Approach

Try every capacity from max(weights) upward, simulate the greedy day-by-day loading, and return the first capacity that finishes within D days.

```
for cap from max(weights) to sum(weights):
    days = daysNeeded(weights, cap)
    if days <= D:
        return cap

function daysNeeded(weights, cap):
    days = 1, load = 0
    for w in weights:
        if load + w > cap:
            days = days + 1
            load = 0
        load = load + w
    return days
```

⌚ **Time:** $O((\text{sum} - \text{max}) * n)$ // trying every capacity, simulating over all packages each time 📁 **Space:** $O(1)$

Optimized Approach

Binary search capacity in [max(weights), sum(weights)]. For each candidate mid capacity, greedily simulate how many days it takes (same daysNeeded logic). If days ≤ D, mid works — try a smaller capacity (right = mid). Otherwise try larger (left = mid + 1).

```
left = max(weights), right = sum(weights)
while left < right:
    mid = left + (right - left) / 2
    days = daysNeeded(weights, mid)    // same greedy simulation as brute force
    if days <= D:
        right = mid                    // mid works, try smaller capacity
    else:
        left = mid + 1                 // mid too small, need more capacity
return left
```

⌚ **Time:** $O(n * \log(\text{sum} - \text{max}))$ // binary search over capacity, $O(n)$ simulation per check 📁 **Space:** $O(1)$

Pattern: Tree BFS

How to identify this pattern

- The input is a tree (usually binary, sometimes n-ary) and the problem cares about LEVELS — depth, level-by-level values, the widest level, the last node of each level, or connections between siblings.
- Keywords to watch for: "level order", "level by level", "zigzag", "level averages", "minimum/maximum depth", "right side view", "connect next pointers".
- The brute force (plain recursive DFS) can technically collect the same data, but requires extra bookkeeping (passing depth down, then grouping results by depth afterward). A queue-based BFS naturally processes one level at a time, which is a more direct match for 'level' questions.
- A queue is the data structure signature of this pattern — you push a node's children onto a queue, and process nodes in the exact order they were discovered, which is what guarantees level-by-level order.
- If the problem instead cares about ROOT-TO-LEAF paths, subtree sums, or 'deepest first' processing, that's Tree DFS, not Tree BFS — level information is the deciding factor.

Q41. Binary Tree Level Order Traversal

Problem Statement

Given the root of a binary tree, return the level order traversal of its nodes' values (i.e., from left to right, level by level).

Example root = [3,9,20,null,null,15,7] → Output: [[3],[9,20],[15,7]]

 **Pattern Insight — how to spot it:** This is the base template for the entire pattern: use a queue seeded with the root. In each iteration of the outer loop, record the CURRENT queue size (this is exactly how many nodes are in the current level) — then pop exactly that many nodes, collecting their values and pushing their children for the next level. This 'snapshot the queue size' trick is what separates level-order BFS from plain BFS.

Brute Force Approach

DFS the tree while tracking depth; append each node's value into a list indexed by its depth (creating new sub-lists as needed). Requires post-processing to preserve left-to-right order, and depth bookkeeping through recursion.

```
result = []
function dfs(node, depth):
    if node == null: return
    if depth == len(result):
        result.append([])
    result[depth].append(node.val)
    dfs(node.left, depth + 1)
    dfs(node.right, depth + 1)
dfs(root, 0)
return result
```

 **Time:** O(n) // visits every node once, though it's DFS pretending to be level-order  **Space:** O(n) // recursion stack + output

Optimized Approach

Standard queue-based BFS: initialize queue with root. While queue is non-empty, record levelSize = queue.length, pop exactly that many nodes into the current level's list (pushing their children as you go), then append that level's list to the result.

```
if root == null: return []
queue = [root]
result = []
while queue is not empty:
    levelSize = len(queue)
    level = []
    for i from 0 to levelSize - 1:
        node = queue.popFront()
        level.append(node.val)
        if node.left: queue.pushBack(node.left)
        if node.right: queue.pushBack(node.right)
    result.append(level)
return result
```

 **Time:** O(n) // each node enqueued and dequeued exactly once  **Space:** O(n) // queue holds up to the widest level; output stores all nodes

Q42. Binary Tree Zigzag Level Order Traversal

Problem Statement

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values (left to right, then right to left for the next level, and alternate).

Example root = [3,9,20,null,null,15,7] → Output: [[3],[20,9],[15,7]]

 **Pattern Insight — how to spot it:** Identical to standard level order BFS — the queue processing and level-size snapshot don't change at all. The only addition is a boolean flag toggled after each level that decides whether to reverse that level's collected list before adding it to the result. This shows how a 'variant' pattern problem often reuses the base template with one small twist.

Brute Force Approach

Do standard level-order DFS/BFS to collect all levels normally, then reverse every other level's list afterward.

```
levels = levelOrderTraversal(root)    // same as the base Tree BFS pattern
for i from 0 to len(levels) - 1:
    if i is odd:
        levels[i] = reverse(levels[i])
return levels
```

 **Time:** O(n)  **Space:** O(n)

Optimized Approach

Run the standard queue-based level order BFS. Maintain a leftToRight boolean, toggled after each level. When appending the current level's list to result, reverse it first if leftToRight is false.

```
if root == null: return []
queue = [root]
result = []
leftToRight = true
while queue is not empty:
    levelSize = len(queue)
    level = []
    for i from 0 to levelSize - 1:
        node = queue.popFront()
        level.append(node.val)
        if node.left: queue.pushBack(node.left)
        if node.right: queue.pushBack(node.right)
    if not leftToRight:
        level = reverse(level)
    result.append(level)
    leftToRight = not leftToRight
return result
```

 **Time:** O(n)  **Space:** O(n)

Q43. Average of Levels in Binary Tree

Problem Statement

Given the root of a binary tree, return the average value of the nodes on each level, as an array.

Example root = [3,9,20,null,null,15,7] → Output: [3, 14.5, 11]

 **Pattern Insight — how to spot it:** Another direct reuse of the base BFS template: instead of collecting each level's node VALUES into a list, just accumulate a running sum and count while draining that level from the queue, then divide.

Brute Force Approach

Collect all levels with standard level-order traversal, then compute the average of each level's list afterward.

```
levels = levelOrderTraversal(root)
result = []
for level in levels:
    result.append(sum(level) / len(level))
return result
```

 **Time:** O(n)  **Space:** O(n) // stores every node's value before averaging

Optimized Approach

Standard queue-based BFS, but instead of storing the level's values, accumulate levelSum and levelCount while draining the queue, then push levelSum / levelCount directly to the result.

```
queue = [root]
result = []
while queue is not empty:
    levelSize = len(queue)
    levelSum = 0
    for i from 0 to levelSize - 1:
        node = queue.popFront()
        levelSum = levelSum + node.val
        if node.left: queue.pushBack(node.left)
        if node.right: queue.pushBack(node.right)
    result.append(levelSum / levelSize)
return result
```

 **Time:** O(n)  **Space:** O(n) // queue only, no need to retain per-node values

Q44. Binary Tree Right Side View

Problem Statement

Given the root of a binary tree, imagine standing on the right side of it; return the values of the nodes you can see, ordered from top to bottom.

Example root = [1,2,3,null,5,null,4] → Output: [1, 3, 4]

 **Pattern Insight — how to spot it:** The 'rightmost visible node per level' is simply the LAST node processed in each level's BFS sweep — so you only need to record the value at the final iteration of the inner loop for each level, no extra bookkeeping needed beyond the standard template.

Brute Force Approach

DFS prioritizing the right child first, recording the first node seen at each new depth (since right-first DFS visits the rightmost node of each level first).

```
result = []
function dfs(node, depth):
    if node == null: return
    if depth == len(result):
        result.append(node.val)
    dfs(node.right, depth + 1)
    dfs(node.left, depth + 1)
dfs(root, 0)
return result
```

 **Time:** O(n)  **Space:** O(n) // recursion stack

Optimized Approach

Standard queue-based BFS. While draining each level, track the value of the LAST node dequeued in that level's inner loop; append only that value to the result once the level finishes.

```
if root == null: return []
queue = [root]
result = []
while queue is not empty:
    levelSize = len(queue)
    rightmost = null
    for i from 0 to levelSize - 1:
        node = queue.popFront()
        rightmost = node.val
        if node.left: queue.pushBack(node.left)
        if node.right: queue.pushBack(node.right)
    result.append(rightmost)
return result
```

 **Time:** O(n)  **Space:** O(n)

Q45. Minimum Depth of Binary Tree

Problem Statement

Given a binary tree, find its minimum depth — the number of nodes along the shortest path from the root down to the nearest LEAF node.

Example root = [3,9,20,null,null,15,7] → Output: 2

 **Pattern Insight — how to spot it:** BFS is strictly better here than DFS: because BFS explores level by level, the FIRST leaf node encountered is guaranteed to be at the minimum depth — you can return immediately without exploring the rest of the tree. A DFS-based solution has no such shortcut and must explore every path to be sure it found the true minimum.

Brute Force Approach

DFS every root-to-leaf path, computing each path's depth, and take the minimum over all of them (has to explore the whole tree even after finding a shallow leaf).

```
function dfs(node):
    if node == null: return infinity
    if node.left == null and node.right == null: return 1
    return 1 + min(dfs(node.left), dfs(node.right))
return dfs(root)
```

 **Time:** $O(n)$ // must explore the entire tree in the worst case  **Space:** $O(h)$ // recursion stack, h = tree height

Optimized Approach

Queue-based BFS. As soon as a dequeued node has no left AND no right child (it's a leaf), its current depth is the answer — return immediately without processing the rest of the queue.

```
if root == null: return 0
queue = [(root, 1)] // (node, depth) pairs
while queue is not empty:
    node, depth = queue.popFront()
    if node.left == null and node.right == null:
        return depth // first leaf found = minimum depth
    if node.left: queue.pushBack((node.left, depth + 1))
    if node.right: queue.pushBack((node.right, depth + 1))
```

 **Time:** $O(n)$ // worst case still $O(n)$, but returns early for shallow leaves — much faster in practice  **Space:** $O(n)$ // queue

Q46. Populating Next Right Pointers in Each Node

Problem Statement

Given a perfect binary tree where every node has an additional 'next' pointer, populate each next pointer to point to its next right node on the same level. Leftmost nextless nodes point to null.

Example root = [1,2,3,4,5,6,7] → next pointers connect 2→3, 4→5→6→7 (each level chained left to right)

 **Pattern Insight — how to spot it:** 'Connect siblings at the same level' is the most literal application of level-order BFS: the inner loop already visits nodes in left-to-right order within a level — just link each node's 'next' to the NEXT node popped in that same inner loop iteration (or null if it's the last one).

Brute Force Approach

Collect all nodes into per-level lists first (standard level order), then do a second pass linking consecutive nodes within each list.

```
levels = levelOrderNodes(root)  // lists of NODE references, not just values
for level in levels:
    for i from 0 to len(level) - 2:
        level[i].next = level[i+1]
    level[-1].next = null
```

 **Time:** O(n)  **Space:** O(n) // storing all node references per level

Optimized Approach

Standard queue-based BFS. While draining a level, link each dequeued node's .next to the node that will be dequeued right after it (peek the queue, or track the 'previous node processed this level').

```
if root == null: return
queue = [root]
while queue is not empty:
    levelSize = len(queue)
    prev = null
    for i from 0 to levelSize - 1:
        node = queue.popFront()
        if prev != null:
            prev.next = node
        prev = node
        if node.left: queue.pushBack(node.left)
        if node.right: queue.pushBack(node.right)
    prev.next = null  // last node in the level
return root
```

 **Time:** O(n)  **Space:** O(n) // queue (can be optimized to O(1) extra space using the next pointers themselves once built, level by level)

Q47. Cousins in Binary Tree

Problem Statement

Given the root of a binary tree with unique values, and two node values x and y, return true if the nodes with those values are cousins (same depth, different parents).

Example root = [1,2,3,4], x = 4, y = 3 → Output: false (different depths)

 **Pattern Insight — how to spot it:** Because 'cousins' is fundamentally a per-level question (same depth) with an extra per-pair check (different parent), BFS naturally fits: process one level at a time, and while draining it, track each node's PARENT alongside its value — if x and y both appear in the same level's sweep but with different recorded parents, they're cousins.

Brute Force Approach

DFS the whole tree tracking (depth, parent) for every node; find x's and y's depth/parent afterward and compare.

```
info = {} // val -> (depth, parent)
function dfs(node, parent, depth):
    if node == null: return
    info[node.val] = (depth, parent)
    dfs(node.left, node, depth + 1)
    dfs(node.right, node, depth + 1)
dfs(root, null, 0)
dx, px = info[x]
dy, py = info[y]
return dx == dy and px != py
```

 **Time:** O(n)  **Space:** O(n) // info map for every node

Optimized Approach

Queue-based BFS storing (node, parent) pairs. While draining each level, record the parent of x and the parent of y if seen in THIS level. After the level finishes, if both were found, they're cousins exactly when their parents differ.

```
queue = [(root, null)]
while queue is not empty:
    levelSize = len(queue)
    parentX, parentY = null, null
    for i from 0 to levelSize - 1:
        node, parent = queue.popFront()
        if node.val == x: parentX = parent
        if node.val == y: parentY = parent
        if node.left: queue.pushBack((node.left, node))
        if node.right: queue.pushBack((node.right, node))
    if parentX != null and parentY != null:
        return parentX != parentY
    if parentX != null or parentY != null:
        return false // only one found at this level → different depths
return false
```

 **Time:** O(n)  **Space:** O(n) // queue

Pattern: Backtracking (Subsets & Permutations)

How to identify this pattern

- You're asked to generate ALL possible subsets, combinations, permutations, or arrangements satisfying some condition — not just count them or find one.
- The problem has a natural 'decision tree' shape: at each step you choose to include/exclude an element, or choose which unused element to place next, and you need to explore every branch.
- Keywords to watch for: "all subsets", "all permutations", "all combinations", "all valid arrangements", "generate all...".
- The brute-force way to enumerate these by hand is exponential no matter what (that's inherent to the problem, not a sign you're missing a trick) — the skill here is building the recursion cleanly: choose → recurse → un-choose (the 'backtrack' step that undoes a choice before trying the next branch).
- A visible sign in the code: a recursive helper function that mutates a shared 'current path' list, recurses deeper, and then POPS/undoes that mutation right after the recursive call returns — that undo step is what makes it 'backtracking' rather than plain recursion.

Q48. Subsets

Problem Statement

Given an integer array 'nums' of unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets.

Example nums = [1,2,3] → Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

 **Pattern Insight — how to spot it:** The canonical backtracking template: at each index, you have exactly two choices — include nums[index] in the current subset, or don't. Recursing through both choices for every index naturally generates every combination of included/excluded elements, which is exactly the power set. Add the current path to the result at EVERY step of the recursion (not just at the leaves), since every partial path is itself a valid subset.

Brute Force Approach

Iteratively build up the power set: start with the empty subset, and for each new number, take every existing subset and create a new copy of it with the number added.

```
result = [[]]
for num in nums:
    newSubsets = []
    for subset in result:
        newSubsets.append(subset + [num])
    result.extend(newSubsets)
return result
```

 **Time:** $O(n * 2^n)$ // 2^n subsets, $O(n)$ to copy each  **Space:** $O(n * 2^n)$ // storing all subsets

Optimized Approach

Backtracking: maintain a 'path' list. At each index, add the current path as a valid subset, then recurse including nums[index] (append it, recurse, then pop it — the backtrack step) before moving to the next index.

```
result = []
path = []
function backtrack(index):
    result.append(copy(path))
    for i from index to n-1:
        path.append(nums[i])
        backtrack(i + 1)
        path.pop() // undo the choice - backtrack
backtrack(0)
return result
```

 **Time:** $O(n * 2^n)$ // same asymptotic result count, but avoids repeated full-list copying overhead of the iterative version  **Space:** $O(n)$ // recursion depth + path, excluding the output itself

Q49. Subsets II (With Duplicates)

Problem Statement

Given an integer array 'nums' that may contain duplicates, return all possible subsets (the power set), without duplicate subsets in the output.

Example nums = [1,2,2] → Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]

 **Pattern Insight — how to spot it:** Same backtracking skeleton as plain Subsets, plus one crucial addition: SORT nums first so duplicates sit next to each other, then at each recursive level, skip a candidate if it equals the PREVIOUS candidate AND the previous one was already considered (not picked) at this same recursion depth. This 'skip same-level duplicates' rule is the standard fix for duplicate-avoidance in backtracking and reappears in many similar problems.

Brute Force Approach

Generate the power set as if there were no duplicates, then dedupe the resulting list of subsets using a set of sorted-tuples.

```
allSubsets = generateAllSubsetsIgnoringDuplicates(nums)    // 2^n subsets, with repeats
seen = empty set
result = []
for subset in allSubsets:
    key = sort(subset)
    if key not in seen:
        seen.add(key)
        result.append(subset)
return result
```

 **Time:** $O(n * 2^n)$ // generate then dedupe  **Space:** $O(n * 2^n)$

Optimized Approach

Sort nums. Backtrack as in plain Subsets, but within the for-loop at each recursion depth, skip nums[i] if $i > \text{startIndex}$ and $\text{nums}[i] == \text{nums}[i-1]$ — this avoids picking 'the same value again' as a fresh branch at the same tree level, which is exactly what creates duplicate subsets.

```
sort(nums)
result = []
path = []
function backtrack(startIndex):
    result.append(copy(path))
    for i from startIndex to n-1:
        if i > startIndex and nums[i] == nums[i-1]:
            continue    // skip duplicate at this recursion level
        path.append(nums[i])
        backtrack(i + 1)
        path.pop()
backtrack(0)
return result
```

 **Time:** $O(n \log n + n * 2^n)$ // sort + backtracking generation, no post-hoc dedup needed  **Space:** $O(n)$ // recursion depth + path, excluding output

Q50. Permutations

Problem Statement

Given an array 'nums' of distinct integers, return all the possible permutations (in any order).

Example nums = [1,2,3] → Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

 **Pattern Insight — how to spot it:** Unlike Subsets (where each element is included-or-not, order doesn't matter), Permutations cares about ORDER, so the recursion structure is different: at each step you pick ANY unused element to place next (not just elements from a fixed starting index onward). A 'used' boolean array (or swapping technique) tracks which elements are already placed in the current path.

Brute Force Approach

Generate all $n!$ orderings by repeatedly inserting each number into every possible position of all permutations built so far.

```
result = [[]]
for num in nums:
    newResult = []
    for perm in result:
        for pos from 0 to len(perm):
            newResult.append(perm[0:pos] + [num] + perm[pos:])
    result = newResult
return result
```

 **Time:** $O(n * n!)$ // $n!$ permutations, $O(n)$ work to build each  **Space:** $O(n * n!)$

Optimized Approach

Backtracking with a 'used' array. At each recursion level, try placing every not-yet-used number next: mark it used, append to path, recurse; after returning, unmark it and pop it (backtrack) before trying the next candidate. A complete path (length == n) is a full permutation, added to the result.

```
result = []
path = []
used = array of n booleans, all false
function backtrack():
    if len(path) == n:
        result.append(copy(path))
        return
    for i from 0 to n-1:
        if used[i]: continue
        used[i] = true
        path.append(nums[i])
        backtrack()
        path.pop()
        used[i] = false
backtrack()
return result
```

 **Time:** $O(n * n!)$ // $n!$ permutations, $O(n)$ work to copy each into the result  **Space:** $O(n)$ // recursion depth + used array, excluding output

Q51. Combinations

Problem Statement

Given two integers n and k , return all possible combinations of k numbers chosen from the range $[1, n]$.

Example $n = 4, k = 2 \rightarrow$ Output: $[[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]$

 **Pattern Insight — how to spot it:** Same 'choose from startIndex onward' backtracking skeleton as Subsets, but instead of recording EVERY partial path, only record paths that reach exactly length k , and stop recursing deeper once that length is hit. A nice optimization ('pruning'): stop the for-loop early once there aren't enough remaining numbers left to reach length k .

Brute Force Approach

Generate all 2^n subsets (as in the Subsets problem) and filter to keep only those with exactly k elements.

```
allSubsets = generateAllSubsets([1..n])
result = []
for subset in allSubsets:
    if len(subset) == k:
        result.append(subset)
return result
```

 **Time:** $O(n * 2^n)$ // generates far more than needed, then filters  **Space:** $O(n * 2^n)$

Optimized Approach

Backtrack from startIndex, adding one number at a time. When path reaches length k , record it and return (don't recurse deeper). Prune: skip starting a branch if there aren't enough numbers left in $[i, n]$ to complete a length- k path.

```
result = []
path = []
function backtrack(start):
    if len(path) == k:
        result.append(copy(path))
        return
    for i from start to n:
        if (n - i + 1) < (k - len(path)):
            break // not enough numbers left – prune
        path.append(i)
        backtrack(i + 1)
        path.pop()
backtrack(1)
return result
```

 **Time:** $O(k * C(n, k))$ // $C(n, k)$ combinations, $O(k)$ to copy each  **Space:** $O(k)$ // recursion depth + path, excluding output

Q52. Combination Sum

Problem Statement

Given a distinct array 'candidates' and a target integer, return all unique combinations where the chosen numbers sum to target. The same number may be chosen an unlimited number of times.

Example candidates = [2,3,6,7], target = 7 → Output: [[2,2,3],[7]]

 **Pattern Insight — how to spot it:** The 'unlimited reuse' requirement is handled with a small but crucial tweak to the standard combinations backtracking template: when recursing after picking candidates[i], recurse starting from index i again (not i+1) — allowing the same number to be picked again — while still starting from i+1 for numbers you've decided to skip, which prevents picking earlier numbers and creating duplicate combinations in a different order.

Brute Force Approach

Explore every possible multiset of candidates up to some depth bound, checking sums — extremely wasteful without pruning by remaining target.

```
// Naive: try all combinations with repetition up to an arbitrary max count,
// checking each one's sum – without pruning by remaining target this explores
// far more branches than necessary and can double-count some paths.
```

 **Time:** Exponential and impractical without target-based pruning  **Space:** O(target) // max recursion depth

Optimized Approach

Backtrack from startIndex with a running remainingTarget. At each step, try including candidates[i] (recurse with the SAME index i, since reuse is allowed, and remainingTarget - candidates[i]); prune whenever remainingTarget goes negative; record the path when remainingTarget hits exactly 0.

```
result = []
path = []
function backtrack(start, remaining):
    if remaining == 0:
        result.append(copy(path))
        return
    if remaining < 0:
        return
    for i from start to len(candidates)-1:
        path.append(candidates[i])
        backtrack(i, remaining - candidates[i])    // same i: reuse allowed
        path.pop()
backtrack(0, target)
return result
```

 **Time:** O(2^{target}) // worst case, bounded by target rather than n due to reuse; well-pruned in practice 
Space: O(target) // max recursion depth

Q53. Word Search

Problem Statement

Given an $m \times n$ grid of characters 'board' and a string 'word', return true if word exists in the grid, formed by sequentially adjacent cells (horizontally or vertically), using each cell at most once.

Example board = [['A','B','C','E'], ['S','F','C','S'], ['A','D','E','E']], word = "ABCCED"
→ Output: true

 **Pattern Insight — how to spot it:** This is backtracking over a GRID instead of an array: from each cell, try all 4 directions as the 'next choice', marking the current cell as visited (temporarily mutating the board or a visited-set) before recursing, and un-marking it on the way back (the backtrack step) so other paths can reuse that cell. Path-finding-with-undo on a grid is a very common backtracking shape worth recognizing on sight.

Brute Force Approach

Conceptually the same idea, but without marking cells visited — this would allow reusing a cell within one path, giving wrong (too permissive) answers; shown to highlight why the 'mark visited, then undo' step is not optional here.

```
// Without a visited marker, the search could reuse the same cell twice
// in one path, which is incorrect per the problem's rules.
// A visited marker (with backtracking cleanup) is required, not optional.
```

 **Time:** N/A — incorrect approach without visited tracking  **Space:** N/A

Optimized Approach

For each starting cell matching word[0], run DFS/backtracking: if the current cell matches word[index], mark it visited (e.g. temporarily overwrite it with a sentinel), recurse into all 4 neighbors for word[index+1], then restore the cell (undo) before returning. Success when index reaches word's length.

```
function backtrack(row, col, index):
    if index == len(word): return true
    if out of bounds or board[row][col] != word[index]: return false
    temp = board[row][col]
    board[row][col] = '#' // mark visited
    found = backtrack(row+1,col,index+1) or backtrack(row-1,col,index+1)
           or backtrack(row,col+1,index+1) or backtrack(row,col-1,index+1)
    board[row][col] = temp // undo - backtrack
    return found

for row in 0..m-1:
    for col in 0..n-1:
        if backtrack(row, col, 0): return true
return false
```

 **Time:** $O(m * n * 4^L)$ // $L = \text{len}(\text{word})$; 4 directions explored at each of L steps, from every starting cell  **Space:** $O(L)$ // recursion depth

Q54. Generate Parentheses

Problem Statement

Given n pairs of parentheses, generate all combinations of well-formed (balanced) parentheses strings.

Example $n = 3 \rightarrow$ Output: ["((()))", "(()())", "(())()", "()(())", "()()()"]

 **Pattern Insight — how to spot it:** Backtracking with CONSTRAINT-BASED pruning instead of a fixed index range: at each step you can add '(' only if you haven't used all n yet, and you can add ')' only if it wouldn't outnumber the '(' placed so far (i.e., $\text{closeCount} < \text{openCount}$). These two guard conditions ensure you only ever explore valid partial strings, which is far more efficient than generating all $2^{(2n)}$ strings and filtering.

Brute Force Approach

Generate all $2^{(2n)}$ strings of length $2n$ made of '(' and ')', then filter to keep only the balanced ones.

```
allStrings = generateAllBinaryStringsOfLength(2 * n, using '(' and ')')
result = []
for s in allStrings:
    if isBalanced(s):
        result.append(s)
return result
```

 **Time:** $O(2^{(2n)} * n)$ // generate then validate each in $O(n)$  **Space:** $O(2^{(2n)} * n)$

Optimized Approach

Backtrack with counts of '(' used (openCount) and ')' used (closeCount). Add '(' if $\text{openCount} < n$; add ')' if $\text{closeCount} < \text{openCount}$. A complete string (length $2n$) is guaranteed balanced by construction — record it.

```
result = []
function backtrack(path, openCount, closeCount):
    if len(path) == 2 * n:
        result.append(path)
        return
    if openCount < n:
        backtrack(path + '(', openCount + 1, closeCount)
    if closeCount < openCount:
        backtrack(path + ')', openCount, closeCount + 1)
backtrack("", 0, 0)
return result
```

 **Time:** $O(4^n / \sqrt{n})$ // bounded by the n th Catalan number — the count of valid strings  **Space:** $O(n)$ // recursion depth, excluding output

Pattern: Top K Elements (Heap)

How to identify this pattern

- The problem asks for the K largest, K smallest, K most/least frequent, or K closest items — any 'top/bottom K' framing.
- You don't need the FULL sorted order, just the K extreme elements — sorting everything ($O(n \log n)$) does more work than necessary when K is much smaller than n.
- Keywords to watch for: "kth largest", "kth smallest", "top k frequent", "k closest", "k most common".
- A (min- or max-) heap of size K is the classic tool: for 'K largest', maintain a MIN-heap of size K (the smallest of your K candidates sits on top, ready to be evicted when a bigger number shows up) — counter-intuitively, you use the OPPOSITE type of heap from what you're looking for, because you want fast access to the weakest current member of your top-K set.
- The brute-force sorts the whole input ($O(n \log n)$); the heap approach processes each element in $O(\log k)$, giving $O(n \log k)$ — a real win when $k \ll n$.

Q55. Kth Largest Element in an Array

Problem Statement

Given an integer array 'nums' and an integer k, return the kth largest element in the array (the kth largest in sorted order, not the kth distinct element).

Example nums = [3,2,1,5,6,4], k = 2 → Output: 5

 **Pattern Insight — how to spot it:** The base template for this whole pattern: maintain a MIN-heap of size k. Push every element; whenever the heap exceeds size k, pop the smallest (evicting the weakest of your current top-k candidates). After processing all elements, the heap's root (minimum of the heap) IS the kth largest overall, since only the k largest elements ever survive in the heap.

Brute Force Approach

Sort the entire array descending and index into position k-1.

```
sortedDesc = sort(nums, descending)
return sortedDesc[k - 1]
```

 **Time:** $O(n \log n)$  **Space:** $O(\log n)$ to $O(n)$ // sort's internal usage

Optimized Approach

Maintain a min-heap. Push each number; if heap size exceeds k, pop the minimum. After processing all n numbers, the heap's top is the kth largest.

```
minHeap = empty min-heap
for num in nums:
    push(minHeap, num)
    if size(minHeap) > k:
        pop(minHeap) // remove the smallest, keeping only the top k
return peek(minHeap) // smallest of the top k = kth largest overall
```

 **Time:** $O(n \log k)$ // each push/pop is $O(\log k)$, done for n elements  **Space:** $O(k)$ // heap holds at most k elements

Q56. Top K Frequent Elements

Problem Statement

Given an integer array 'nums' and an integer k, return the k most frequent elements, in any order.

Example nums = [1,1,1,2,2,3], k = 2 → Output: [1, 2]

 **Pattern Insight — how to spot it:** Two-phase pattern: first build a frequency map (a hash map pass, $O(n)$), THEN apply the min-heap-of-size-k technique from the base problem — but on (frequency, value) pairs instead of raw values, keeping the k pairs with the highest frequency.

Brute Force Approach

Count frequencies with a hash map, then sort all distinct elements by frequency descending and take the first k.

```
freq = frequency map of nums
sortedByFreq = sort(freq.keys(), by freq[key] descending)
return sortedByFreq[0 : k]
```

 **Time:** $O(n + d \log d)$ // d = number of distinct elements, dominated by sorting them  **Space:** $O(n)$ // frequency map

Optimized Approach

Build the frequency map. Push (frequency, value) pairs into a min-heap ordered by frequency, evicting the minimum whenever the heap exceeds size k. The final heap holds the k most frequent elements.

```
freq = frequency map of nums
minHeap = empty min-heap ordered by frequency
for value, count in freq:
    push(minHeap, (count, value))
    if size(minHeap) > k:
        pop(minHeap)
return [value for (count, value) in minHeap]
```

 **Time:** $O(n + d \log k)$ // n to build freq map, d distinct elements pushed/popped in $O(\log k)$ each  **Space:** $O(n + k)$ // frequency map + heap

Q57. K Closest Points to Origin

Problem Statement

Given an array of points on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$, in any order.

Example `points = [[1,3],[-2,2]]`, $k = 1 \rightarrow$ Output: `[[-2,2]]`

 **Pattern Insight — how to spot it:** 'K closest' is 'K smallest by distance' — the same min-heap-of-size-k idea as Kth Largest, but flipped: since you want the SMALLEST distances, use a MAX-heap of size k instead (evict the FARTHEST point whenever the heap grows past k), so the heap always retains the k closest points seen so far.

Brute Force Approach

Compute each point's squared distance to origin, sort all points by that distance, and take the first k .

```
distances = [(x*x + y*y, [x,y]) for [x,y] in points]
sortedByDist = sort(distances, ascending)
return [point for (d, point) in sortedByDist[0:k]]
```

 **Time:** $O(n \log n)$  **Space:** $O(n)$

Optimized Approach

Maintain a max-heap of size k , ordered by squared distance. Push each point's $(\text{distance}, \text{point})$; if heap size exceeds k , pop the maximum (farthest point). The heap ends up holding exactly the k closest points.

```
maxHeap = empty max-heap ordered by distance
for [x, y] in points:
    dist = x*x + y*y
    push(maxHeap, (dist, [x, y]))
    if size(maxHeap) > k:
        pop(maxHeap) // remove the farthest
return [point for (dist, point) in maxHeap]
```

 **Time:** $O(n \log k)$  **Space:** $O(k)$

Q58. Kth Largest Element in a Stream

Problem Statement

Design a class that, given an integer k , initializes with a stream of integers and supports `add(val)` which appends `val` to the stream and returns the k th largest element in the stream so far.

Example $k = 3$, initial nums = [4,5,8,2]; `add(3) → 4`; `add(5) → 5`; `add(10) → 5`; `add(9) → 8`

 **Pattern Insight — how to spot it:** This turns the batch Kth Largest problem into an ONLINE (incremental) one: instead of rebuilding a heap from scratch on every `add()`, KEEP a persistent min-heap of size k across calls. Each `add()` is just: push the new value, evict the min if size $> k$, and the top is always the current answer — $O(\log k)$ per call instead of $O(n \log n)$ per call.

Brute Force Approach

Store all numbers seen so far in a list; on every `add()`, append and re-sort the whole list to find the k th largest.

```
stream = initial nums
function add(val):
    stream.append(val)
    sortedDesc = sort(stream, descending)
    return sortedDesc[k - 1]
```

 **Time:** $O(n \log n)$ per `add()` call // n = current stream size, re-sorted every time  **Space:** $O(n)$ // storing the whole stream

Optimized Approach

Maintain a persistent min-heap of size k across all calls (built once from the initial numbers, keeping only the top k). On each `add()`: push the new value, evict the min if size exceeds k , then return the heap's top.

```
minHeap = empty min-heap
for num in initialNums:
    push(minHeap, num)
    if size(minHeap) > k:
        pop(minHeap)

function add(val):
    push(minHeap, val)
    if size(minHeap) > k:
        pop(minHeap)
    return peek(minHeap)
```

 **Time:** $O(\log k)$ per `add()` call // heap persists between calls  **Space:** $O(k)$ // heap only ever holds k elements

Q59. Find K Pairs with Smallest Sums

Problem Statement

Given two integer arrays 'nums1' and 'nums2' both sorted ascending, and an integer k, return the k pairs (u, v) with u from nums1 and v from nums2 that have the smallest sums.

Example nums1 = [1,7,11], nums2 = [2,4,6], k = 3 → Output: [[1,2],[1,4],[1,6]]

 **Pattern Insight — how to spot it:** Because BOTH arrays are sorted, you don't need to generate all $n \cdot m$ pairs to find the smallest k sums — a max-heap of size k (same eviction idea as K Closest Points) lets you prune aggressively: seed the heap with the most promising early candidates (small indices from nums1 paired with nums2[0]) and expand outward only from pairs already in the heap, since any pair with a larger index in either array can only have an equal-or-larger sum.

Brute Force Approach

Generate every possible pair (u, v) from nums1 x nums2, compute sums, sort by sum, and take the first k.

```
pairs = []
for u in nums1:
    for v in nums2:
        pairs.append([u, v])
sortedPairs = sort(pairs, by sum ascending)
return sortedPairs[0 : k]
```

 **Time:** $O(n \cdot m \cdot \log(n \cdot m))$ // generates and sorts all pairs  **Space:** $O(n \cdot m)$

Optimized Approach

Use a min-heap seeded with pairs (nums1[i], nums2[0]) for i from 0 to $\min(k, \text{len}(\text{nums1})) - 1$, each tagged with its (i, j) indices. Repeatedly pop the smallest-sum pair into the result; for each popped pair (i, j), push (nums1[i], nums2[j+1]) if j+1 is valid. Stop after k pairs are extracted.

```
minHeap = empty min-heap ordered by (nums1[i] + nums2[j])
for i from 0 to  $\min(k, \text{len}(\text{nums1})) - 1$ :
    push(minHeap, (i, 0))
result = []
while minHeap is not empty and  $\text{len}(\text{result}) < k$ :
    i, j = pop(minHeap) // smallest current sum
    result.append([nums1[i], nums2[j]])
    if j + 1 <  $\text{len}(\text{nums2})$ :
        push(minHeap, (i, j + 1))
return result
```

 **Time:** $O(k \log k)$ // at most k pops, each triggering one $O(\log k)$ push  **Space:** $O(k)$ // heap size bounded by k

Q60. Top K Frequent Words

Problem Statement

Given an array of strings 'words' and an integer k, return the k most frequent strings, sorted by frequency (highest first); words with the same frequency are ordered alphabetically.

Example words = ["i","love","leetcode","i","love","coding"], k = 2 → Output: ["i", "love"]

 **Pattern Insight — how to spot it:** Same frequency-map + heap skeleton as Top K Frequent Elements, but the heap's comparison rule now needs a TIE-BREAKER: order primarily by frequency, and alphabetically (reversed, since it's a min-heap being used to keep the 'worst' k candidates evictable) when frequencies match. This shows how the base pattern extends cleanly to compound sort criteria.

Brute Force Approach

Count frequencies with a hash map, then sort all distinct words by (frequency descending, word ascending) and take the first k.

```
freq = frequency map of words
sortedWords = sort(freq.keys(), by (freq[word] descending, word ascending))
return sortedWords[0 : k]
```

 **Time:** $O(d \log d)$ // d = number of distinct words  **Space:** $O(n)$ // frequency map

Optimized Approach

Build the frequency map. Use a min-heap ordered by (frequency ascending, word DESCENDING — so alphabetically 'larger' words are evicted first on ties, leaving the 'smaller' ones which come first alphabetically). Evict when size exceeds k, then reverse-extract into final order.

```
freq = frequency map of words
minHeap = empty min-heap ordered by (freq[word] asc, word desc)
for word, count in freq:
    push(minHeap, (count, word))
    if size(minHeap) > k:
        pop(minHeap)
result = extract all from minHeap, sort by (count desc, word asc)
return result
```

 **Time:** $O(d \log k)$ // d distinct words, $O(\log k)$ per heap operation  **Space:** $O(n + k)$

Q61. Kth Smallest Element in a Sorted Matrix

Problem Statement

Given an $n \times n$ matrix where each row and column is sorted in ascending order, return the k th smallest element in the matrix.

Example matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8 → Output: 13

 **Pattern Insight — how to spot it:** Because each ROW is individually sorted, you never need to look at a row's later elements before its earlier ones — this is a K-way merge in disguise. Seed a min-heap with the first element of each row; repeatedly pop the smallest and push the next element from THAT SAME row. After k pops, you've extracted the k th smallest overall, since the heap always represents the current 'frontier' of the smallest unexplored elements across all rows.

Brute Force Approach

Flatten the entire matrix into a single list, sort it, and index into position $k-1$.

```
flat = []
for row in matrix:
    flat.extend(row)
sortedFlat = sort(flat)
return sortedFlat[k - 1]
```

 **Time:** $O(n^2 \log(n^2))$ // n^2 total elements, sorted  **Space:** $O(n^2)$

Optimized Approach

Min-heap seeded with $(\text{matrix}[i][0], i, 0)$ for each row i . Pop the smallest $k-1$ times, each time pushing the next element from the same row ($\text{col}+1$) if it exists. The k th pop is the answer.

```
minHeap = empty min-heap ordered by value
for i from 0 to n-1:
    push(minHeap, (matrix[i][0], i, 0))
for count from 1 to k:
    val, row, col = pop(minHeap)
    if count == k:
        return val
    if col + 1 < n:
        push(minHeap, (matrix[row][col+1], row, col + 1))
```

 **Time:** $O(k \log n)$ // at most k pops, heap size bounded by n (one entry per row)  **Space:** $O(n)$ // heap holds at most one entry per row

Q62. Task Scheduler

Problem Statement

Given a char array 'tasks' (each letter represents a task) and a non-negative integer n representing the cooldown period between two same tasks, return the minimum number of CPU intervals required to finish all tasks (idle intervals allowed).

Example tasks = ["A","A","A","B","B","B"], n = 2 → Output: 8 (A B idle A B idle A B)



Pattern Insight — how to spot it: A frequency-driven, GREEDY use of a max-heap: always schedule the currently most-frequent remaining task next (to spread out the task that will run out of 'room' soonest), using the heap to always know which task has the highest remaining count. A cooldown queue tracks tasks temporarily benched until their cooldown expires, after which they're pushed back into the heap.

Brute Force Approach

Simulate minute-by-minute, at each step scanning all task counts to pick the valid task with the highest remaining count that isn't on cooldown — a linear scan per minute instead of a heap lookup.

```
counts = frequency map of tasks
time = 0
cooldownUntil = map of task -> earliest available time, all 0
remaining = total tasks left
while remaining > 0:
    best = task with max counts[task] where cooldownUntil[task] <= time (linear scan)
    if best exists:
        counts[best] -= 1
        remaining -= 1
        cooldownUntil[best] = time + n + 1
    time += 1
return time
```

⌚ **Time:** $O(\text{total_time} * 26)$ // linear scan over task types at every minute 📁 **Space:** $O(26) = O(1)$

Optimized Approach

Max-heap of task counts. Repeatedly pop up to n+1 of the most frequent tasks per 'cycle', decrementing their counts and holding them in a cooldown queue; push tasks back onto the heap once their cooldown period passes. Count total intervals used (including idle slots when the heap empties before a cycle finishes but tasks remain).

```
counts = frequency map of tasks
maxHeap = counts.values() as a max-heap
time = 0
cooldownQueue = empty queue of (count, availableAtTime)
while maxHeap is not empty or cooldownQueue is not empty:
    time = time + 1
    if maxHeap is not empty:
        count = pop(maxHeap) - 1
        if count > 0:
            cooldownQueue.push((count, time + n))
    if cooldownQueue.front() exists and cooldownQueue.front().availableAtTime == time:
        count, _ = cooldownQueue.pop()
        push(maxHeap, count)
return time
```

🕒 **Time:** $O(\text{total_time} * \log 26) = O(\text{total_time})$ // heap of at most 26 task types 📁 **Space:** $O(26) = O(1)$

Pattern: Monotonic Stack

How to identify this pattern

- You need the NEXT (or previous) greater/smaller element for every position in an array — the classic giveaway phrase.
- The problem involves comparing each element to its neighbors to the left/right to find a boundary, a span, or an area (histogram-style problems, temperature-waiting-day problems, stock span).
- The brute force compares every pair of elements ($O(n^2)$) — for each position, scanning forward/backward until a bigger/smaller element is found. A monotonic stack processes each element in amortized $O(1)$, giving $O(n)$ overall.
- The stack invariant is the key idea: keep the stack's elements in strictly increasing (or decreasing) order at all times. When a new element would violate that order, POP elements off the stack until it's restored — and each pop is itself an answer (e.g. 'this popped element's next greater element is the current one').
- Keywords to watch for: "next greater element", "next smaller element", "daily temperatures", "histogram", "stock span", "remove k digits to make smallest".

Q63. Next Greater Element I

Problem Statement

Given two arrays 'nums1' and 'nums2' (nums1 is a subset of nums2, both with unique elements), for each element in nums1, find its next greater element in nums2 (the first greater element to its right in nums2); return -1 if none exists.

Example nums1 = [4,1,2], nums2 = [1,3,4,2] → Output: [-1, 3, -1]

 **Pattern Insight — how to spot it:** The base template for the whole Monotonic Stack pattern: scan nums2 left to right, maintaining a DECREASING stack of 'still waiting for their next greater element'. When the current number is bigger than the stack's top, that's the answer for everything popped off — pop and record until the stack top is bigger (or empty), then push the current number.

Brute Force Approach

For each number in nums2, scan forward from its position until a bigger number is found (or the end is reached).

```
nextGreater = {}
for i from 0 to len(nums2)-1:
    nextGreater[nums2[i]] = -1
    for j from i+1 to len(nums2)-1:
        if nums2[j] > nums2[i]:
            nextGreater[nums2[i]] = nums2[j]
            break
return [nextGreater[x] for x in nums1]
```

 **Time:** $O(m * n)$ // $m = \text{len}(\text{nums1})$, $n = \text{len}(\text{nums2})$, inner scan for each element  **Space:** $O(n)$ // map

Optimized Approach

Single pass over nums2 with a decreasing monotonic stack (holding numbers whose next-greater hasn't been found yet). For each number, while it's bigger than the stack's top, pop and record the current number as that top's next greater. Push the current number. Finally look up each nums1 element in the map (default -1).

```
nextGreater = {}
stack = [] // decreasing stack of values
for num in nums2:
    while stack is not empty and stack.top() < num:
        nextGreater[stack.pop()] = num
    stack.push(num)
// anything left in stack has no next greater → nextGreater default -1
return [nextGreater.get(x, -1) for x in nums1]
```

 **Time:** $O(m + n)$ // each element pushed and popped at most once (amortized $O(1)$ per element)  **Space:** $O(n)$ // stack + map

Q64. Next Greater Element II (Circular Array)

Problem Statement

Given a circular integer array 'nums' (the next element of the last element is the first element), return an array where the value at each index is the next greater element, considering the circular wraparound; return -1 if it doesn't exist.

Example nums = [1,2,1] → Output: [2, -1, 2] (last 1 wraps to find 2 at the front)

 **Pattern Insight — how to spot it:** The 'circular' twist is handled with a simple trick: conceptually iterate through the array TWICE (using index $i \% n$), applying the exact same decreasing-stack logic as Next Greater Element I. The second pass lets elements near the end 'see' elements near the beginning, simulating wraparound, without physically duplicating the array in memory (just use $i \% n$ to index).

Brute Force Approach

For each index, scan forward up to $n-1$ additional steps (wrapping with modulo) looking for a bigger element.

```
n = len(nums)
result = [-1] * n
for i from 0 to n-1:
    for offset from 1 to n-1:
        j = (i + offset) % n
        if nums[j] > nums[i]:
            result[i] = nums[j]
            break
return result
```

 **Time:** $O(n^2)$  **Space:** $O(n)$ // output

Optimized Approach

Same decreasing monotonic stack as Next Greater Element I, but iterate i from 0 to $2n-1$, using index $i \% n$ to read values. Only push indices (not values) during the first pass through — after n steps, stop pushing new indices but keep popping to resolve remaining ones with wraparound values.

```
n = len(nums)
result = [-1] * n
stack = [] // stack of INDICES, decreasing by value
for i from 0 to 2*n - 1:
    num = nums[i % n]
    while stack is not empty and nums[stack.top()] < num:
        result[stack.pop()] = num
    if i < n:
        stack.push(i)
return result
```

 **Time:** $O(n)$ // $2n$ iterations, each index pushed/popped at most once  **Space:** $O(n)$ // stack + output

Q65. Daily Temperatures

Problem Statement

Given an array of daily temperatures, return an array 'answer' where answer[i] is the number of days you have to wait after day i to get a warmer temperature; if no such day exists, answer[i] = 0.

Example temperatures = [73,74,75,71,69,72,76,73] → Output: [1,1,4,2,1,1,0,0]

 **Pattern Insight — how to spot it:** This is Next Greater Element, but instead of recording the VALUE of the next greater element, you record the DISTANCE (in days) to it — a trivial modification once you recognize the underlying pattern: store INDICES on the stack (not values) so you can compute currentIndex - poppedIndex when resolving each entry.

Brute Force Approach

For each day, scan forward until a warmer day is found, counting the number of days waited.

```
answer = [0] * n
for i from 0 to n-1:
    for j from i+1 to n-1:
        if temperatures[j] > temperatures[i]:
            answer[i] = j - i
            break
return answer
```

 **Time:** $O(n^2)$  **Space:** $O(1)$ // excluding output

Optimized Approach

Decreasing monotonic stack of INDICES (by temperature). For each day, while the current temperature is higher than the temperature at the stack's top index, pop it and set answer[poppedIndex] = currentIndex - poppedIndex. Push the current index.

```
answer = [0] * n
stack = [] // stack of indices, decreasing by temperature
for i from 0 to n-1:
    while stack is not empty and temperatures[stack.top()] < temperatures[i]:
        j = stack.pop()
        answer[j] = i - j
    stack.push(i)
return answer
```

 **Time:** $O(n)$ // each index pushed and popped at most once  **Space:** $O(n)$ // stack + output

Q66. Largest Rectangle in Histogram

Problem Statement

Given an array of integers 'heights' representing the histogram's bar heights (each bar has width 1), return the area of the largest rectangle that can be formed within the histogram.

Example heights = [2,1,5,6,2,3] → Output: 10 (rectangle of height 5, width 2 spanning bars [5,6])

 **Pattern Insight — how to spot it:** For every bar, the largest rectangle using that bar's height is bounded by the nearest SHORTER bar on each side (its height can't extend past a shorter neighbor). That's exactly 'previous smaller element' and 'next smaller element' — a monotonic INCREASING stack processes this in one pass: whenever a new bar is shorter than the stack's top, the popped bar's rectangle can be finalized because its right boundary (the current shorter bar) is now known, and its left boundary is whatever remains below it on the stack.

Brute Force Approach

For each bar, expand left and right as far as possible while the height stays $\geq$ the current bar's height, computing the resulting rectangle's area.

```
maxArea = 0
for i from 0 to n-1:
    left = i
    while left > 0 and heights[left-1] >= heights[i]: left -= 1
    right = i
    while right < n-1 and heights[right+1] >= heights[i]: right += 1
    maxArea = max(maxArea, heights[i] * (right - left + 1))
return maxArea
```

 **Time:** $O(n^2)$ // worst case, expanding for every bar  **Space:** $O(1)$

Optimized Approach

Increasing monotonic stack of indices. For each bar, while it's shorter than the height at the stack's top, pop and compute that popped bar's rectangle: height = heights[popped], width = current index - (new stack top index) - 1 (or currentIndex if stack empties). Push the current index. After the scan, drain any remaining stack entries treating 'n' as the right boundary.

```
stack = [] // indices, increasing by height
maxArea = 0
for i from 0 to n: // one extra iteration with height 0 to flush the stack
    currentHeight = heights[i] if i < n else 0
    while stack is not empty and heights[stack.top()] > currentHeight:
        h = heights[stack.pop()]
        width = i if stack is empty else i - stack.top() - 1
        maxArea = max(maxArea, h * width)
    stack.push(i)
return maxArea
```

 **Time:** $O(n)$ // each index pushed and popped at most once  **Space:** $O(n)$ // stack

Q67. Trapping Rain Water

Problem Statement

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example `height = [0,1,0,2,1,0,1,3,2,1,2,1]` → Output: 6

 **Pattern Insight — how to spot it:** A monotonic (non-increasing) stack of indices representing 'walls waiting for a taller wall to their right to trap water against them'. When a bar taller than the stack's top appears, pop the top as the 'floor' of a water pocket — the trapped water's height is bounded by the SHORTER of the current bar and the new stack top (the left wall), and its width is the horizontal gap between them. (Note: this problem also has a well-known two-pointer $O(1)$ -space solution, but the stack approach directly generalizes the 'resolve on a taller bar' logic used across this whole pattern.)

Brute Force Approach

For each bar, find the tallest bar to its left and to its right; the water trapped above that bar is $\min(\text{leftMax}, \text{rightMax}) - \text{height}[i]$, if positive.

```
total = 0
for i from 0 to n-1:
    leftMax = max(height[0..i])
    rightMax = max(height[i..n-1])
    total += max(0, min(leftMax, rightMax) - height[i])
return total
```

 **Time:** $O(n^2)$ // recomputing leftMax/rightMax by scanning for every bar  **Space:** $O(1)$

Optimized Approach

Monotonic non-increasing stack of indices. For each bar, while it's taller than the height at the stack's top, pop that index as the 'basin floor'; the water level there is $\min(\text{current height}, \text{new stack top's height})$ minus the floor's height, times the width between them. Push the current index.

```
stack = [] // indices, non-increasing by height
total = 0
for i from 0 to n-1:
    while stack is not empty and height[stack.top()] < height[i]:
        floor = stack.pop()
        if stack is empty: break // no left wall, can't trap here
        leftWall = stack.top()
        boundedHeight = min(height[i], height[leftWall]) - height[floor]
        width = i - leftWall - 1
        total += boundedHeight * width
    stack.push(i)
return total
```

 **Time:** $O(n)$ // each index pushed and popped at most once  **Space:** $O(n)$ // stack

Q68. Remove K Digits

Problem Statement

Given a non-negative integer as a string 'num' and an integer k, remove k digits from num so that the resulting number (as a string, no leading zeros, unless it's "0") is the smallest possible.

Example num = "1432219", k = 3 → Output: "1219"

 **Pattern Insight — how to spot it:** To make the smallest possible number, you want smaller digits as far LEFT as possible. This is a monotonic INCREASING stack of digits: scan left to right, and whenever the current digit is smaller than the stack's top AND you still have removals (k) left, pop the top (remove that larger digit) — greedily preferring the smaller current digit earlier in the result.

Brute Force Approach

Try every combination of k digits to remove (or every resulting string after removing k digits), compare all resulting numbers, and keep the smallest.

```
best = null
for each way to choose (len(num) - k) digits to KEEP in order:
    candidate = those digits joined, stripped of leading zeros (or "0")
    if best is null or candidate < best (numerically/lexicographically with equal length):
        best = candidate
return best
```

 **Time:** $O(C(n, n-k) * n)$ // exponential — combinatorial choice of digits to keep  **Space:** $O(n)$

Optimized Approach

Increasing monotonic stack of digits. For each digit, while $k > 0$ and the stack's top is bigger than the current digit, pop it (remove it) and decrement k. Push the current digit. After the scan, if $k > 0$ remains, remove that many digits from the END (the stack is already non-decreasing, so the end has the largest remaining digits). Strip leading zeros.

```
stack = [] // digits, increasing
for digit in num:
    while k > 0 and stack is not empty and stack.top() > digit:
        stack.pop()
        k = k - 1
    stack.push(digit)
while k > 0: // still need to remove — take from the end
    stack.pop()
    k = k - 1
result = join(stack).stripLeadingZeros()
return result if result != "" else "0"
```

 **Time:** $O(n)$ // each digit pushed and popped at most once  **Space:** $O(n)$ // stack

Q69. Online Stock Span

Problem Statement

Design an algorithm that collects daily stock prices and returns the SPAN of the stock's price for the current day — the number of consecutive days (including today) the price has been less than or equal to today's price.

Example prices in order: 100, 80, 60, 70, 60, 75, 85 → spans: 1, 1, 1, 2, 1, 4, 6

 **Pattern Insight — how to spot it:** The span for today is exactly 'today's index minus the index of the nearest previous day with a HIGHER price' — a 'previous greater element' problem, solved online (one call at a time) with a monotonic DECREASING stack of (price, span) pairs. When a new price arrives, pop and ABSORB the span of every stack entry with a price $\leq$ the new price (since those days are all part of today's streak too), then push (newPrice, accumulatedSpan).

Brute Force Approach

On each new price, scan backward through all previously seen prices, counting consecutive days with price $\leq$ today's, until a higher price or the start is reached.

```
prices = []
function next(price):
    prices.append(price)
    span = 0
    i = len(prices) - 1
    while i >= 0 and prices[i] <= price:
        span += 1
        i -= 1
    return span
```

 **Time:** $O(n)$ per call in the worst case $\rightarrow O(n^2)$ total  **Space:** $O(n)$ // all prices stored

Optimized Approach

Maintain a stack of (price, span) pairs, decreasing by price. On each new price, initialize span = 1, and while the stack's top has price $\leq$ new price, pop it and add its span to the running total (absorbing that whole streak). Push (newPrice, span) and return span.

```
stack = [] // (price, span) pairs, decreasing by price
function next(price):
    span = 1
    while stack is not empty and stack.top().price <= price:
        _, poppedSpan = stack.pop()
        span = span + poppedSpan
    stack.push((price, span))
    return span
```

 **Time:** $O(1)$ amortized per call // each entry pushed and popped at most once overall  **Space:** $O(n)$ // stack, worst case one entry per call

Q70. Asteroid Collision

Problem Statement

Given an array of integers 'asteroids' representing asteroids in a row (absolute value = size, sign = direction: positive right, negative left), simulate collisions: when two collide, the smaller explodes; if equal, both explode. Return the state of the asteroids after all collisions.

Example asteroids = [5,10,-5] → Output: [5, 10] asteroids = [8,-8] → Output: []

 **Pattern Insight — how to spot it:** A stack naturally models this because collisions only ever happen between a RIGHT-moving asteroid already 'in flight' (pushed on the stack) and a new LEFT-moving asteroid arriving — you process left to right, and whenever the top of the stack is moving right (positive) and the current asteroid moves left (negative), they collide: pop/compare sizes in a loop until either the current asteroid is destroyed, the stack top is destroyed, or no more collisions are possible (stack top moves left, or current asteroid still alive and stack empty/moving away).

Brute Force Approach

Repeatedly scan the whole array looking for any adjacent colliding pair (right-moving followed by left-moving), resolve that one collision, and rescan from the start — restart after each single collision.

```
arr = copy(asteroids)
changed = true
while changed:
    changed = false
    for i from 0 to len(arr)-2:
        if arr[i] > 0 and arr[i+1] < 0:
            resolve the collision between arr[i] and arr[i+1] (remove the loser, or both
            if equal)
            changed = true
            break
return arr
```

 **Time:** $O(n^2)$ // repeated full rescans after each collision  **Space:** $O(n)$

Optimized Approach

Single left-to-right pass with a stack. For each asteroid: if it's moving right, or the stack is empty, or the stack's top is already moving left (no possible collision), push it. Otherwise (stack top is positive/right-moving and current is negative/left-moving), repeatedly resolve the collision: pop the smaller of the two; if sizes are equal, pop the top and discard the current asteroid too; continue checking the new stack top against the (still surviving) current asteroid.

```
stack = []
for a in asteroids:
    alive = true
    while alive and a < 0 and stack is not empty and stack.top() > 0:
        if stack.top() < -a:
            stack.pop() // top asteroid destroyed, keep checking
        else if stack.top() == -a:
            stack.pop() // both destroyed
            alive = false
        else:
            alive = false // current asteroid destroyed, top survives
    if alive:
```

```
        stack.push(a)  
    return stack
```

🕒 **Time:** $O(n)$ // each asteroid pushed and popped at most once  **Space:** $O(n)$ // stack